ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Corso di Laurea Triennale in
Ingegneria Informatica

Progettazione di Applicazioni Web basate su Rust e WebAssembly

Relatore
Prof. Paolo Bellavista

Tesi di Laurea di
Cristian Davide Conte

Anno Accademico 2020-2021
Sessione I

Indice

1 Introduzione **3**

2 WebAssembly **8**

2.1 Architettura interna 9
2.2 Esecuzione nel browser 11
2.3 Caratteristiche principali 13
2.4 Confronto WebAssembly - Javascript 16

3 Rust **22**

3.1 Caratteristiche principali 23
3.2 Gestione della memoria 26
3.3 Runtime 30
3.4 Confronto Rust - Go - Java 37

4 Applicazione Web in Rust **41**

4.1 Backend 43
4.2 Frontend 50
4.3 Esecuzione e Test 56

Capitolo 1

Introduzione

Nonostante le prime riflessioni sul concetto di ipertesto, inteso come un insieme di documenti messi in relazione tra loro tramite collegamenti unidirezionali [1], risalgano al 1945: anno in cui Vannevar Bush, direttore dell' *"Office of Scientific Research and Development"*, le presentò per la prima volta sulla rivista *"The Atlantic"* in un articolo intitolato *"As We May Think"* [2]; il World Wide Web ha origini ben più recenti.
È infatti nel 1989 che al CERN l'informatico britannico Sir Tim Berners-Lee propose, all'interno di un documento intitolato *"Information Management: A Proposal"* [3], la creazione di un progetto volto alla condivisione e distribuzione su rete Internet di documenti statici in forma ipertestuale.
Due anni più tardi il primo sito Web venne messo online e nel 1993 la tecnologia alla base del World Wide Web fu resa pubblica: nacque così il Web 1.0.

La sua diffusione fu rapidissima ma presto ci si accorse dell'impossibilità di servire contenuti dinamicamente generati e fu perciò necessario sviluppare nuove tecnologie per sopperire a tale mancanza.
Una delle prime soluzioni, diventata poi standard de facto del Web dinamico delle origini, fu la Common Gateway Interface, comunemente nota come CGI.
La sua caratteristica principale era quella di permettere la comunicazione tra applicazioni esterne, scritte in un qualunque linguaggio, ed un server Web.
Questa comunicazione poteva avvenire tramite le variabili d'ambiente del sistema operativo del server Web, tramite parametri su riga di comando (comune con l'utilizzo del metodo GET di HTTP), oppure attraverso standard input (con il metodo POST di HTTP); permettendo di generare dinamicamente contenuto a seconda dei parametri passati.

---

[1] https://en.wikipedia.org/wiki/Hyperlink
[2] https://www.theatlantic.com/magazine/archive/1945/07/as-we-may-think/303881/
[3] https://cds.cern.ch/record/369245/files/dd-89-001.pdf

Nonostante CGI fosse un passo in avanti verso la creazione di pagine Web modulari e generate sulla base delle necessità dell'utente, era una soluzione che presentava problemi di prestazioni, robustezza e portabilità del codice; per questo motivo furono introdotte nuove soluzioni, tra cui un'evoluzione di CGI chiamata *"FastCGI"*, che hanno concorso ad evolvere il concetto di World Wide Web portandolo a diventare a tutti gli effetti quello che oggi è conosciuto come il Web 2.0.

Tra le soluzioni che hanno trovato maggiore successo vi sono le Java Applet, così come Java Servlet e le JavaServer Pages affiancate dal crescente sviluppo di linguaggi orientati al Web tra cui PHP e Javascript.
Quest'ultimo è diventato negli anni il linguaggio di programmazione più utilizzato sul Web, grazie anche al supporto nativo da parte dei moderni browser, portando alla creazione di innumerevoli framework e tecnologie, delle quali è alla base, con lo scopo di espandere le funzionalità del nuovo Web dinamico.
Ad oggi Javascript è quindi considerato lo standard de facto per la realizzazione di applicazioni Web, ma ciò non vuol dire che lo sviluppo di software orientato alla produzione di applicazioni Web si sia arrestato.

Il 21 Marzo 2013, Mozilla Foundation ha presentato un progetto chiamato *"asm.js"*: un sottoinsieme di istruzioni Javascript che poteva essere utilizzato come target di compilazione di basso livello e che forniva un'astrazione simile a quella della macchina virtuale C/C++ [4].
Il 17 Giugno 2015 è stato annunciato per la prima volta WebAssembly: *"un formato di codice di basso livello, sicuro e portatile caratterizzato da una notazione compatta che permette l'esecuzione efficiente di codice"* [5].
WebAssembly, noto anche come WASM, permette di compilare ed eseguire codice di basso livello sul Web ed è basato sul progetto *"asm.js"*: iniziali implementazioni sfruttavano perfino le stesse API del progetto di Mozilla Foundation. A differenza del suo predecessore, non è però un sottoinsieme di istruzioni Javascript ed ha come obiettivo quello di essere un *Instruction Set Architecture* standard che non presuppone come piattaforma di esecuzione il browser: codice WebAssembly, in futuro, dovrebbe poter essere eseguito su qualsiasi piattaforma ne supporti l'ISA.

---

[4] http://asmjs.org/spec/latest/
[5] https://webassembly.github.io/spec/core/intro/introduction.html

Nonostante inizialmente supportasse solamente la compilazione dai linguaggi C e C++, dal 2017, anno in cui è stato rilasciato l'MVP WASM [6] ed è stato aggiunto il supporto a questa tecnologia da parte di tutti i principali browser moderni, è possibile compilare anche altri linguaggi di programmazione in codice WebAssembly: tra questi troviamo Rust.

Rust è un linguaggio di programmazione di basso livello, con typing statico e di tipo imperativo, nato nel 2006 all'interno del Mozilla Research Group come progetto personale di un dipendente dell'azienda.
Mozilla Foundation ha poi finanziato e sviluppato Rust che è stato presentato ufficialmente nel 2009 per poi venire lanciato nella sua prima versione stabile nel 2015.
Rust trae ispirazione sia da linguaggi di basso livello come C, C++ e Smalltalk sia linguaggi di più alto livello come Python e Ruby ed ha tra i suoi obiettivi principali l'efficienza e la memory safety.

Grazie ad API Rust che aggiungono il supporto all'ISA WebAssembly è quindi possibile utilizzare questo linguaggio su piattaforme per cui non era stato inizialmente progettato, come il browser, ed osservare gli interessanti risvolti di questo approccio che è alla base di un nuovo modo di concepire lo sviluppo di applicazioni Web.
Uno dei vantaggi di questo nuovo metodo è proprio la portabilità del codice che rispetto alle classiche applicazioni Web scritte in HTML e Javascript, permette di non dover riscrivere completamente un progetto software quando viene eseguito in ambienti diversi da quello per cui era stato originariamente progettato a patto che questi supportino l'ISA WASM.
Un altro caso d'uso di questa nuova tecnologia, reso possibile dalle altissime prestazioni del codice WebAssembly, è rappresentato da tutte quelle applicazioni che necessitano di una potenza computazionale che il codice Javascript non è in grado di gestire efficientemente.
Un esempio di applicazione, originariamente scritta in C++, che ha trovato nuova vita sul Web grazie a WebAssembly è *"Google Earth"*.
La nota applicazione per l'osservazione di località sparse per tutto il pianeta nasceva l'11 giugno 2001 col nome di *"Keyhole Earthviewer"* ed era un'applicazione nativa per il sistema operativo Microsoft Windows, successivamente riscritta per supportare Linux, Android, OSX, e iOS.

---

[6] https://en.wikipedia.org/wiki/Minimum_viable_product

Nel 2017 è stata rilasciata la versione Web di Google Earth che presentava però forti limitazioni dovute alle prestazioni relativamente scarse che questa versione poteva offrire su macchine non eccessivamente prestanti.
Il 20 giugno 2019 è stata infine rilasciata una nuova versione Web che sfruttava WASM come target di compilazione della versione desktop e che offriva prestazioni all'incirca diciannove volte superiori a quelle della precedente versione scritta in Javascript e HTML.

Nonostante i risultati in termini di prestazioni siano migliori utilizzando WebAssembly e non Javascript in quei casi in cui sia necessario effettuare operazioni computazionalmente complesse client-side; questo nuovo formato non deve essere trattato come una polvere magica che rende qualsiasi software più efficiente: la velocità d'esecuzione pura è solo uno dei tanti obiettivi di questo nuovo formato e perdipiù, come verrà approfondito nel capitolo 2.4, esistono situazioni in cui codice Javascript risulta essere più performante dell'equivalente WebAssembly.
L'ISA WASM permette anche di utilizzare questa tecnologia per eseguire codice che verrebbe normalmente ritenuto insicuro sfruttando le caratteristiche della virtual stack machine, che è alla base del funzionamento di WebAssembly, e l'ambiente di esecuzione chiuso all'interno del quale questo codice viene eseguito.
"webassembly.sh" [7] è un progetto WASM che sfrutta a pieno questa peculiare caratteristica proponendo un terminale, simile alla shell Bash dei sistemi Unix, all'interno del browser e che supporta il caricamento dinamico di moduli WASM esterni, eventualmente provenienti dal filesystem locale dell'utente, senza particolari limitazioni.

Come anticipato precedentemente è possibile scrivere applicazioni Web tramite linguaggi di basso livello, come Rust, ed eseguirli sul Web tramite WASM in alternativa alle storiche tecnologie HTML e Javascript; questo nuovo approccio è particolarmente utile se si considerano i dispositivi mobili e le risorse limitate di cui questi dispongono rispetto a macchine desktop e server.
In particolare è stato osservato che smartphone e tablet possono impiegare dai 20 ai 40 secondi solamente per il parsing di applicazioni Javascript: la situazione si aggrava ulteriormente se si considerano anche i tempi di download dei file stessi.

---

[7] https://webassembly.sh/

Il codice WebAssembly, che contiene molte meno istruzioni essendo compilato e non interpretato, non necessita di parsing, ma deve comunque essere sottoposto a decodifica. Il supporto da parte dei browser a WebAssembly include la possibilità di decodificare nativamente codice WASM fino a venti volte più velocemente rispetto al processo di parsing delle equivalenti istruzioni Javascript.
Combinando questa funzionalità alla modularità del nuovo formato ed al supporto alle WebAPI per lo streaming dei file WebAssembly, si osserva che, anche in presenza di una velocità di connessione limitata, i tempi necessari affinché le applicazioni Web possano essere utilizzate dagli utenti sono estremamente ridotti rispetto alla controparte Javascript.

Alcuni esempi di progetti di rilievo che hanno adottato la tecnologia WebAssembly traendone benefici sono: *"TensorFlow.js"* [8], una libreria Javascript per il training e lo sviluppo di modelli per il machine learning, che ha sfruttato WebAssembly all'interno del suo backend; *"lichess.org"* [9], uno dei più importanti siti di scacchi, che ha usato WASM per realizzare un port di Stockfish [10], uno dei motori scacchistici migliori al mondo, reso poi utilizzabile dagli utenti per l'analisi delle partite in tempo reale direttamente sul browser; *"Unity"* [11], un motore grafico cross-platform, recentemente reso disponibile sul Web grazie a WebAssembly e alle API WebGL; *"Figma"* [12], uno dei principali strumenti per il design di interfacce grafiche 2D, che praticamente senza alcuno sforzo ha effettuato il porting sul Web dell'applicazione originale scritta in C++.

---

[8] https://www.tensorflow.org/
[9] https://lichess.org/
[10] https://github.com/official-stockfish/Stockfish
[11] https://unity.com/
[12] https://www.figma.com/

Capitolo 2

WebAssembly

Spiegare il funzionamento di WebAssembly è un compito semplice ed allo stesso tempo terribilmente complesso a causa della doppia natura di questo nuovo formato. Da un lato abbiamo le idee alla base di WASM che riprendono i noti concetti del linguaggio Assembly [13] e della Java Virtual Machine [14], dall'altro troviamo la forma e le convenzioni della versione testuale che, al fine di rendere il codice chiaro e più comprensibile all'uomo, introducono formalismi sintattico-lessicali che appesantiscono notevolmente le nozioni. Lo scopo di questo capitolo è quello di fornire una conoscenza dei principi fondamentali di WebAssembly senza però affrontare nello specifico tutti i formalismi sintattici con cui essi sono rappresentati.

Di seguito viene riportato un esempio, in formato testuale, di codice WebAssembly per la stampa a video della stringa "Hello World".

```
(module
  (type $FUNCSIG$ii (func (param i32) (result i32)))
  (type $FUNCSIG$iii (func (param i32 i32) (result i32)))
  (import "env" "printf" (func $printf (param i32 i32) (result i32)))
  (table 0 anyfunc) (memory $0 1) (data (i32.const 16) "Hello World\00")
  (export "memory" (memory $0)) (export "main" (func $main))
  (func $main (; 1 ;) (result i32)
    (drop
     (call $printf
      (i32.const 16)
      (i32.const 0)
     )
    ) (i32.const 0)
  )
)
```

[13] https://en.wikipedia.org/wiki/Assembly_language
[14] https://docs.oracle.com/javase/specs/jvms/se7/html/jvms-1.html#jvms-1.2

2.1 Architettura interna

Il funzionamento dell'ISA WebAssembly è quello di una stack machine virtuale che accetta solamente due tipologie di istruzioni chiamate rispettivamente *"Simple Instruction"* e *"Control Instruction"*.
In questo caso per "stack machine" si intende una macchina virtuale le cui due operazioni principali, chiamate *"push"* e *"pop"*, inseriscono e rimuovono istruzioni dalla cima di una struttura dati astratta di tipo LIFO, chiamata *"stack"*[15].
Le *"Simple Instruction"* sono istruzioni, utilizzabili nei moduli WASM, che prelevano un argomento dalla cima dello *"stack"* tramite l'operazione di *"pop"* e che successivamente eseguono operazioni di tipo aritmetico ottenendo un risultato di cui poi viene fatto il *"push"* sullo *"stack"* della virtual machine WASM. Le *"Control Instruction"* eseguono invece operazioni di controllo di flusso (if, loop, branch, return ed altre ancora) causando, in caso di situazioni critiche, la creazione di interrupt sincroni chiamati *"trap"* che vengono poi propagati all'host all'interno del quale il modulo WASM è in esecuzione.

I moduli WebAssembly rappresentano il codice, presentato in un formato binario o testuale, che verrà eseguito a runtime e sono costituiti da un header seguito da una sequenza di sezioni. Nell'header sono presenti informazioni riguardanti la versione di WebAssembly utilizzata ed una sequenza di byte particolare, chiamata *"magic_cookie"*, che segnala all'ambiente host in cui il codice WASM verrà messo in esecuzione che quel file deve essere trattato come modulo WebAssembly. Le sezioni contengono le dichiarazioni di variabili e funzioni, definiscono le risorse utilizzate dal codice e descrivono il comportamento di quest'ultimo in relazione ad altri moduli se necessario.
Prima di poter essere eseguiti, i moduli WASM vengono validati dall'ambiente host, ovvero ci si assicura che la sintassi e la semantica delle istruzioni siano corrette. Successivamente i moduli vengono allocati ed istanziati riservando uno spazio nella memoria heap, chiamato *"Linear Memory"*, necessario all'allocazione di eventuali strutture dati. Questo spazio è un array di byte, diviso in porzioni di 64 KiB chiamate *"pages"*, che può solamente crescere in dimensioni e mai diminuire e che può essere manipolato dal codice WebAssembly tramite operazioni di *"load"* e *"store"*.

---

[15] https://en.wikipedia.org/wiki/Stack_(abstract_data_type)

L'ISA WASM permette di condividere una stessa istanza di *"Linear Memory"* tra moduli diversi e supporta istruzioni specifiche per garantire che gli accessi ad una stessa *"page"* siano coordinati.
Nonostante questa specifica sia ancora in fase di sviluppo a causa della nuova natura del formato WebAssembly e di alcune polemiche riguardanti la sicurezza di questo tipo di operazione all'interno di un browser Web; combinando le funzionalità della *"Linear Memory"* di WASM ai WebWorkers [16] offerti nativamente dai browser, WebAssembly dovrebbe poter fornire supporto a funzionalità tipiche di ambienti di programmazione concorrenti come i thread.

[16] https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Using_web_workers

2.2 Esecuzione nel browser

Nonostante l'ISA WebAssembly non specifichi che l'ambiente di esecuzione del codice , chiamato *"host environment"*, debba per forza essere un browser; la stragrande maggioranza del codice WASM viene eseguito proprio all'interno di questa piattaforma e ciò è possibile grazie al supporto nativo da parte dei principali browser Web moderni.
Questo supporto viene garantito principalmente grazie a due componenti software: una virtual machine in grado di eseguire istruzioni WASM ed un insieme di API, chiamate *"WebAPI"*, che ogni browser espone al fine di offrire maggiore controllo su quelle funzionalità dei dispositivi che sarebbero altresì di basso livello e difficili da sfruttare. Esempi di queste API sono: le API DOM per la manipolazione di elementi all'interno di pagine Web, le API WebGL che permettono di effettuare il rendering di interfacce grafiche 2D o 3D interattive, le API IndexedDB per l'accesso a database esterni e molte altre.

Le VM dei browser sono state storicamente progettate per eseguire solamente codice Javascript per via del suo largo utilizzo nella maggior parte dei componenti che fanno parte del Web moderno. Da Marzo 2017 è stato però aggiunto il supporto all'esecuzione di codice WebAssembly.
Queste virtual machine sono in grado di eseguire le istruzioni dell'ISA WASM grazie al fatto che il codice scritto in questo formato viene prima compilato nel corrispondente formato binario, creando il modulo WASM equivalente, e poi istanziato come oggetto nella memoria delle VM.
Poichè sia Javascript sia WASM possono essere eseguiti da una stessa macchina virtuale, l'interoperabilità tra i due tipi di codice è possibile.
WebAssembly non si pone infatti come obiettivo quello di sostituire Javascript bensì cerca di integrarsi con codice già esistente al fine di migliorare prestazioni, affidabilità e portabilità.
Codice WASM può accedere alle stesse WebAPI a cui il codice Javascript equivalente può accedere, rendendo possibile scrivere vere e proprie applicazioni Web in WebAssembly per poi eseguirle sul browser come se fossero state scritte in Javascript.
Nel caso in cui del codice WASM venisse eseguito su un browser che non supporta ancora questo nuovo formato, viene effettuata un'operazione di *"polyfill"* tramite la quale il codice WASM viene tradotto nell'equivalente Javascript tramite l'utilizzo di *"asm.js"* di cui è stato discusso nel capitolo 1.

Attualmente l'utilizzo di WebAssembly all'interno dei browser ha alcune limitazioni. Ad esempio, nonostante l'interazione tra WebAssembly e le WebAPI sia possibile, questa non può avvenire in modo diretto e deve appoggiarsi al linguaggio Javascript per le invocazioni dei metodi. Un'altra limitazione del formato WebAssembly è rappresentata dal fatto che al momento questo formato non supporta alcun tipo di Garbage Collector e non è quindi possibile compilare ed eseguire sul Web quei linguaggi che ne fanno uso, come ad esempio Java e C#. Anche se questo fosse possibile però, gli strumenti per il debugging sul Web di applicazioni scritte in linguaggi di basso livello e poi compilate in WASM, sono ad uno stato embrionale ed ancora in fase di sperimentazione: individuare problemi all'interno di invocazioni di chiamate a funzioni innestate risulterebbe praticamente impossibile: questo perché quando viene generata una trap dal codice WASM, il debugger del browser indica la linea del codice WebAssembly che ha generato l'interrupt, ma non fornisce alcuna indicazione riguardo all'ubicazione di tale linea all'interno del codice originale.
Inoltre attualmente esistono solamente quattro tipi di dati e sono tutti tipi numerici (i32, i64, f32 e f64) il che vuol dire che ogni altro tipo di dato deve essere serializzato all'interno della Linear Memory ed utilizzato tramite un riferimento alla sua posizione all'interno di della memoria WASM portando a svantaggi in termini di performance e di chiarezza del codice.

Quelli appena elencati sono comunque tutti problemi noti dell'ISA WASM per i quali sono già state proposte alcune soluzioni e che quindi potrebbero essere corretti nell'imminente futuro.

2.3 Caratteristiche principali

L'Instruction Set Architecture WebAssembly, di base, non fornisce alcun supporto per l'interazione con elementi esterni all'host environment.
Questo significa che operazioni come quelle di Input/Output, di accesso a file e risorse esterne oppure l'invocazione di procedure del sistema operativo sono a carico dell'ambiente in cui il codice WASM viene messo in esecuzione:
nel caso dei browser Web, come visto nel capitolo precedente, queste tipologie di operazioni sono garantite dalle WebAPI.
Ciò garantisce un elevato grado di sicurezza che rappresenta uno dei punti cardine attorno al quale WebAssembly è stato sviluppato: basti anche solo pensare che qualsiasi accesso non consentito al di fuori della Linear Memory provoca la creazione di una trap che viene propagata all'esterno, esattamente come quando una qualsiasi istruzione prova ad accedere ad uno spazio di memoria non riservato ad istruzioni della virtual stack machine WASM.

Nonostante siano scelte progettuali estremamente limitanti nei confronti dell'ISA, esse permettono di garantire una maggiore separazione dei ruoli tra ambiente host e codice hosted e al contempo rendono il comportamento del codice WebAssembly totalmente deterministico e quindi prevedibile già in fase di compilazione.
Quest'ultima è fondamentale per capire un'altra delle idee alla base della creazione di questo nuovo formato: il codice WASM viene infatti compilato in codice macchina nativo (ovvero in istruzioni x86, x64 e ARM) rendendo i tempi di esecuzione molto bassi.
Nel caso dei browser Web le prestazioni del codice WASM sono solitamente ordini di grandezza maggiori rispetto alla controparte Javascript.
Il motivo dietro a questa differenza di velocità è da attribuire al modo in cui la virtual machine dei browser Web tratta i due diversi tipi di codice.
Javascript è un linguaggio interpretato e non ha quindi la possibilità di effettuare quelle ottimizzazioni che il compilatore WebAssembly esegue e c'è perciò bisogno di una fase iniziale in cui i Javascript engine (Chrome V8, Firefox SpiderMonkey e Safari WebKit) osservano il codice e cercano di effettuare delle assunzioni che permettano di ottimizzare i tempi di esecuzione: in Google Chrome questa fase è chiamata *"Ignition"*.

Il codice WebAssembly non ha bisogno di questa fase poiché le eventuali ottimizzazioni sono state effettuate ancor prima che il modulo venisse caricato: la fase iniziale in questo caso viene chiamata *"Liftoff"*.
Una volta che entrambi i codici hanno terminato la fase iniziale si entra nella fase del *V8 engine* chiamata *"TurboFan"* in cui il browser è in grado di sfruttare le ottimizzazioni fatte in precedenza in combinazione alla cache della CPU per offrire il massimo grado di prestazioni. Con nomenclature differenti, lo stesso discorso può essere applicato al Firefox SpiderMonkey e al Safari WebKit.
Nonostante le ottimizzazioni effettuate, il codice WebAssembly risulta comunque essere più efficiente.
Di seguito vengono riportati i risultati di un benchmark basato sull'implementazione dell'algoritmo di ordinamento BubbleSort [17], eseguito su un array di decine di migliaia di entry, scritto in Javascript e nell'equivalente WASM.

Linguaggio	Fase V8 engine	Tempi di Esecuzione	vs JS TurboFan
Javascript	Ignition	2340.07ms	22.5x
WASM	Liftoff	256.98ms	2.5x
Javascript	TurboFan	103.85ms	1.0x
WASM	TurboFan	65.42ms	0.6x

Un'altra interessante caratteristica del formato WebAssembly è che quest'ultimo non è in alcun modo dipendente dall'hardware o dal sistema operativo su cui viene messo in esecuzione ed offre quindi un ventaglio di opportunità più ampio rispetto ad un linguaggio di programmazione convenzionale legato a poche piattaforme specifiche. WASM non è però un linguaggio di programmazione e la portabilità del codice WebAssembly diviene utile, come già anticipato nel capitolo 1, specialmente se combinata al fatto che questo formato può essere utilizzato come target di compilazione da altri linguaggi: compilare, su una stessa piattaforma, un qualsiasi linguaggio supportato in WASM produrrà sempre lo stesso risultato in termini di comportamento e prestazioni.

---

[17] https://en.wikipedia.org/wiki/Bubble_sort

Oltre a ciò, utilizzando WebAssembly come formato comune è possibile mettere in comunicazione linguaggi di programmazione che normalmente sarebbero incompatibili traendo vantaggio dalla Linear Memory condivisa tra i vari moduli. Questi ultimi sono, tra l'altro, scomponibili in frammenti di codice più piccoli, i quali possono essere codificati, trasmessi dalla sorgente in modo seriale o parallelo, decodificati dall'ambiente host in cui poi verranno messi in esecuzione ed infine validati ed inizializzati non appena completamente disponibili: proprio come quello che avviene quando si fa una richiesta HTTP di tipo GET ad un server Web che rende disponibili applicazioni Web disponibili tramite WebAssembly.

2.4 Confronto WebAssembly - Javascript

Così come l'introduzione di Javascript, all'interno del processo di creazione di applicazioni Web nell'ormai lontano 1995, rappresentò una rivoluzione per il Web dell'epoca; allo stesso modo la realizzazione di un nuovo formato di codice standard come WebAssembly potrebbe rappresentare un punto di svolta per lo sviluppo di applicazioni Web moderne.
Prestazioni, portabilità del codice, dimensioni dei file ridotte, sono alcune tra le caratteristiche principali di questo nuovo formato che però non si pone di sostituire Javascript.
Le ragioni dietro questa scelta sono molteplici: in primis la nota impossibilità per qualsiasi tecnologia neonata di soppiantare completamente ed in breve tempo la diretta concorrente a causa del largo utilizzo di quest'ultima all'interno di una vastità incredibile di sistemi. Quindi anche se WASM si fosse proposto come nuovo standard per la programmazione all'interno del Web moderno la sua adozione sarebbe sicuramente stata limitata rispetto a Javascript. In più WebAssembly non dispone di un ecosistema di framework, librerie e software di supporto che la controparte Javascript ha accumulato negli anni: i software che utilizzano questa nuova tecnologia, seppur in aumento, sono ancora limitati ed in numero relativamente ridotto.
Una fondamentale caratteristica di WASM che, almeno per il momento, rende definitivamente impossibile la sua adozione come nuovo punto di riferimento per lo sviluppo sul Web è rappresentata dal modo in cui i moduli WebAssembly vengono caricati nella virtual machine dei browser moderni. Questo processo necessita obbligatoriamente dell'utilizzo di API Javascript per l'istanziazione del modulo e la creazione della Linear Memory.
Inoltre gli strumenti per il debug e lo sviluppo sul Web di applicazioni scritte in linguaggi di basso livello e poi compilate in WASM, sono limitati e non ancora paragonabili a quelli disponibili per lo sviluppo di software Javascript; ciò tiene lontano questo nuovo formato dalla maggior parte degli ambienti di sviluppo software professionali in cui il tempo di sviluppo è una risorsa preziosa.

Oltretutto uno dei motivi principali per cui si dovrebbe preferire WebAssembly a Javascript sono le migliori performance in termini di tempo di esecuzione; esistono però casi in cui queste prestazioni potrebbero non risultare migliori.

Per dimostrarlo vengono riportati due benchmark relativi ai tempi di esecuzione (espressi in operazioni al secondo) di codice scritto in Rust poi compilato in WASM confrontato con l'equivalente Javascript, che mostrano risultati contrastanti.

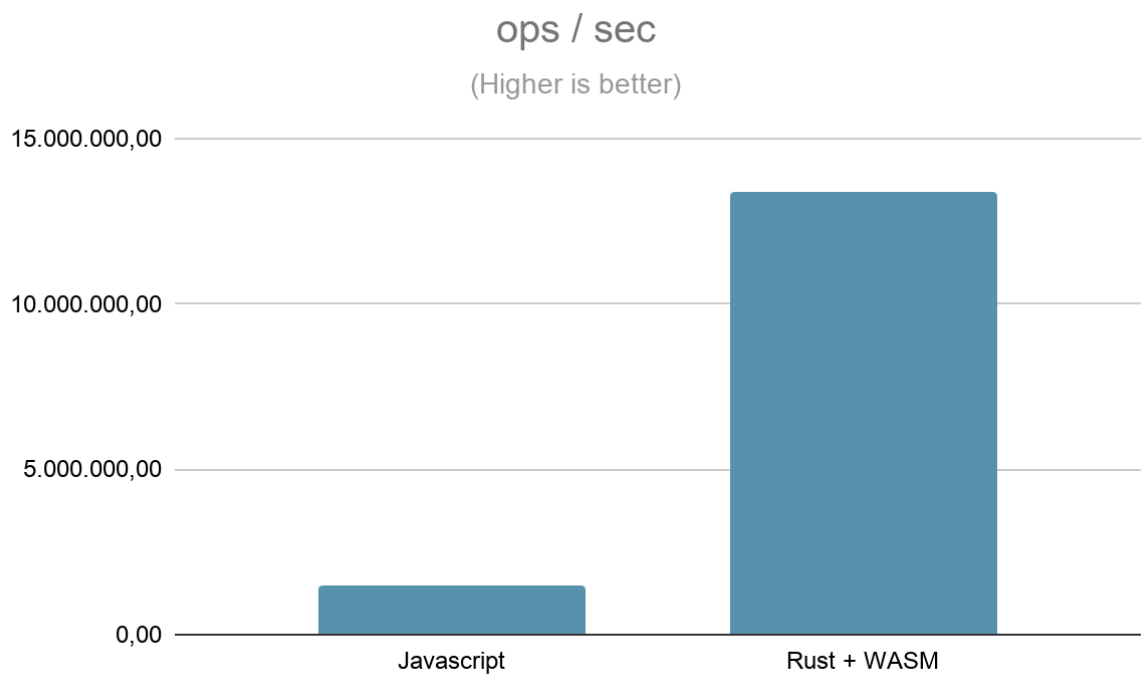
Il primo test vuole misurare la velocità di esecuzione relativa ad operazioni in cui viene utilizzata estensivamente la CPU.

Input: max = 100000	

`fn count(max: i32) -> i32 {`
`  let mut result = 0;`
`  for i in 0..max {`
`  result = result + 1;`
`  }`
`  result`
`}`
	`function count(max) {`
	`  let result = 0;`
	`  for (let i = 0; i < max; i++) {`
	`  result++;`
	`  }`
	`  return result;`
	`}`

Rust	**Javascript**

Di seguito i risultati ottenuti.



Il codice Rust compilato in WASM risulta essere estremamente più performante dell'equivalente Javascript grazie al fatto che il compilatore WebAssembly ha potuto effettuare delle ottimizzazioni che l'engine Javascript deve eseguire a runtime. Ad esempio, nel semplice benchmark riportato sopra, analizzando il comportamento del codice all'interno del ciclo for, ci si accorge che in realtà non è affatto necessario eseguire quest'ultimo per conoscere il valore di ritorno della funzione *"count"*: esso sarà sempre uguale al valore *"max"* passato in input.
Inoltre, dato che Rust e WASM sono staticamente tipizzati, la VM WebAssembly non effettua operazioni di inferenza del tipo che invece troviamo durante il runtime del codice Javascript, ad esempio, al fine di valutare se l'espressione *"i < max"* risulta sensata (lo è solo se il parametro *"max"* è un numero).

Tenendo presenti queste considerazioni, vengono riportate due versioni modificate della funzione *"count"* che riportano più fedelmente le operazioni concretamente effettuate a runtime dai due codici e che meglio evidenziano l'origine dell'enorme differenza di prestazioni riscontrata.

Input: max = 100000	

`fn count(max: i32) -> i32 {`
`  max`
`}`
	`function count(max) {`
	`  let result = 0;`
	`  let parsedMax = parseInt(max);`
	`  if (typeof parsedMax !== "number") {`
	`  return result;`
	`  }`
	`  for (let i = 0; i <parsedMax; i++) {`
	`  result++;`
	`  }`
	`  return result;`
	`}`

Rust **(After Compiler Optimizations)**	**Javascript** **(With Explicit Type Coercion)**

Eseguendo un numero elevato di test, gli engine Javascript dei browser riuscirebbero ad ottimizzare il codice in maniera pressoché identica al compilatore WASM, ciò richiederebbe però del tempo a runtime che invece la controparte WebAssembly ha già speso in fase di compilazione.
La combinazione *"linguaggio di basso livello + WebAssembly"* porterebbe incredibili benefici a quelle applicazioni che necessitano di eseguire nel modo più efficiente possibile grandi quantità di calcoli in un ammontare di tempo ridotto.
È il caso delle simulazioni fisiche in tempo reale nelle quali sono comuni situazioni in cui è necessario calcolare frequentemente la posizione di oggetti nello spazio, facendo uso di grandi quantità di operazioni tra matrici e calcoli vettoriali notoriamente complessi dal punto di vista computazionale.
Un altro caso d'uso per le migliori prestazioni di WebAssembly, in termini di CPU, è rappresentato dalle applicazioni di editing video-grafico. In questo ambito è comune infatti che si manipoli un'enorme quantità di pixel in un lasso di tempo ristretto. Per esempio, ruotare un'immagine in 4k di 90° equivale a manipolare contemporaneamente la posizione di 8.294.400 pixel, operazione che può impiegare fino ad 8 secondi se eseguita su dispositivi mobili non troppo prestanti; la situazione si aggrava se ad essere ruotata non è solo un'immagine, ma un insieme di frame all'interno di un video.
I dispositivi mobili sono di fatto l'applicazione ideale per le performance WASM; a dimostrarlo è lo studio effettuato da eBay [18] durante la creazione di un lettore di barcode per la loro applicazione Web: la funzionalità sviluppata utilizzando WebAssembly si è dimostrata addirittura 50 volte più veloce della controparte Javascript permettendo loro, assieme ad altre ottimizzazioni, di utilizzare la nuova funzione all'interno della loro applicazione Web.

[18] https://tech.ebayinc.com/engineering/webassembly-at-ebay-a-real-world-use-case/

Ora si vuole prendere in considerazione un ulteriore esempio nel quale si desidera misurare la velocità di esecuzione (espressa in operazioni al secondo) relativa ad operazioni in cui viene utilizzata estensivamente la RAM.

Input: str1 = **"Hello World"** str2 = **"World"**	

`fn myIncludes(str1: &str, str2: &str ) -> bool {`
`  str1.contains(str2)`
`}`
	`function myIncludes(str1, str2) {`
	`  return str1.includes(str2);`
	`}`

Rust	**Javascript**

Di seguito i risultati ottenuti.

In questo caso il codice Javascript risulta enormemente più performante rispetto alla controparte Rust/WebAssembly.

Nel benchmark sopra riportato vengono manipolate e confrontate delle stringhe e ciò richiede alcuni passaggi non evidenti da parte del codice WebAssembly; in particolare il motivo del risultato ottenuto è dovuto al modo in cui il codice Rust ed il modulo WASM comunicano.
Dato che, come precedentemente discusso, WASM riconosce solamente tipi di dato numerici è necessario che le stringhe Rust vengano codificate (ad esempio in UTF-16, ovvero in una sequenza di numeri a 16 bit) prima di poter essere utilizzate; successivamente è necessario che il modulo WebAssembly ottenga una porzione di memoria libera all'interno della Linear Memory in cui poter salvare la stringa, eventualmente allocando nuovo spazio.
La stringa deve essere poi copiata dalla memoria heap di Javascript, attraverso il quale avviene la gestione della memoria dei moduli WebAssembly, all'interno dell'array di byte della Linear Memory: solo allora la stringa potrà essere utilizzata dal modulo.
Nonostante il codice Rust nativo sia effettivamente più efficiente dell'equivalente Javascript nativo, le operazioni di codifica dei dati (o serializzazione nel caso di strutture dati complesse) e di copia all'interno dell'array di memoria WASM, rallentano notevolmente l'esecuzione del codice rispetto alla controparte Javascript.

I risultati di questi semplici test evidenziano che la velocità di esecuzione pura non è sufficiente a rappresentare un valido motivo per cui WebAssembly possa effettivamente sostituire Javascript all'interno dello sviluppo di applicazioni Web ed inoltre mettono in risalto l'aspetto collaborativo che, a questo punto dello sviluppo del nuovo formato, è necessario considerare: al momento esistono situazioni in cui WASM può portare incredibili benefici ad alcuni aspetti dello sviluppo software per il Web (ad esempio la gestione di operazioni CPU-intensive), ma questi non costituiscono la totalità dei contesti ed è difatti ancora possibile utilizzare efficacemente Javascript nella maggior parte dei casi (ad esempio nella manipolazione del DOM, di stringhe e array).

Capitolo 3

Rust

Rust è un linguaggio di programmazione di basso livello ideato da Mozilla, che ha posto al centro del processo di sviluppo la risoluzione di problemi ricorrenti nella creazione e gestione del software; tra questi ultimi troviamo la gestione sicura della memoria e il controllo accurato e granulare di componenti hardware e software di basso livello al fine di ottenere prestazioni elevate.
Rust è molto apprezzato dalla comunità degli sviluppatori, tant'è vero che è stato votato come linguaggio più amato dal 2016 ad oggi su StackOverflow [19] e l'interesse verso questa tecnologia è in continuo aumento [20].
Aziende del calibro di Microsoft hanno speso parole di supporto nei confronti di Rust indicandolo come migliore alternativa, attualmente disponibile, a linguaggi come C e C++ per via della predicibilità delle performance offerte da questo linguaggio [21].
Amazon è un'altra grande azienda che supporta Rust, infatti parte dell'infrastruttura AWS ed alcuni progetti futuri dell'azienda includono parti basate su questo interessante linguaggio.
Dal 2019, Amazon è inoltre diventata uno degli sponsor ufficiali di Rust [22] unendosi ad altri colossi del settore come Microsoft, ARM e Google.
Quest'ultima azienda ha, negli ultimi anni, sviluppato un prototipo di sistema operativo completamente open source chiamato *"Fuchsia"* [23] che si basa sui principi di sicurezza e privacy ed è stato realizzato tramite Rust.
Rust è inoltre uno dei principali linguaggi di programmazione supportati dal nuovo formato WebAssembly, di cui è stato discusso nel capitolo precedente, e che ha subito un interessante percorso di sviluppo portandolo ad essere uno dei linguaggi più amati dagli sviluppatori, in grado di poter essere utilizzato sulla piattaforma più vasta di distribuzione di contenuti e software attualmente esistente: il Web.

---

[19] https://insights.stackoverflow.com/survey/2020
[20] https://insights.stackoverflow.com/trends?tags=rust
[21] https://msrc-blog.microsoft.com/2019/07/22/why-rust-for-safe-systems-programming/
[22] https://aws.amazon.com/it/blogs/opensource/aws-sponsorship-of-the-rust-project/
[23] https://fuchsia.dev/

3.1 Caratteristiche principali

Rust è un linguaggio con typing statico ed imperativo; ogni variabile viene dichiarata con la keyword *"let"* e la dichiarazione implica l'assegnamento a quella variabile di un tipo specifico immutabile che può essere omesso ad eccezione delle situazioni in cui il compilatore Rust non è in grado di inferirlo.
Il valore contenuto all'interno delle variabili non è alterabile a meno che lo sviluppatore non utilizzi la parola chiave *"mut"* nelle loro dichiarazioni: questa keyword comunque non permette di cambiare il tipo che il compilatore Rust ha assegnato loro in fase di compilazione.

Di seguito vengono forniti due esempi di sintassi di dichiarazioni di variabili rispettivamente mutabili ed immutabili.

`fn main() {`
`  let mut my_var : i64 = 0;`
`  my_var += 1; //OK !`
`  println!("{}", my_var); //Prints "1"`
`}`
	`fn main() {`
	`  let my_var : i64 = 0;`
	`  my_var += 1; //ERR !`
	`  /* Program aborts */`
	`}`

Mutable Variable	**Immutable Variable**

Rust supporta lo shadowing di variabili, ovvero la capacità di una variabile di oscurarne un'altra con lo stesso nome all'interno dello stesso scope.
Questa caratteristica permette di ridefinire come mutabili variabili inizialmente dichiarate come immutabili e di modificarne quindi il valore.
In Rust, lo shadowing permette inoltre di variare il tipo di dato rappresentato da una variabile precedentemente dichiarata nonostante questa operazione sia normalmente vietata.
Ciò è consentito perché non appena il compilatore Rust incontra una definizione di una variabile, sia il tipo che il valore precedentemente contenuti in essa cessano di esistere: in pratica lo shadowing in questo linguaggio agisce al pari di una nuova dichiarazione ed è permanente.
In linguaggi come Java o C++, a differenza di quanto accade in Rust, è necessario definire scope differenti per ogni nuova dichiarazione di una variabile avente lo stesso nome identificativo di una precedentemente dichiarata, rendendo più verbosa, e soprattutto temporanea, l'operazione di shadowing.

Di seguito viene riportato un confronto tra lo shadowing in Rust e in Java.

`fn main() {`
`  let my_var = "Hello World";`
`  let mut my_var : i64 = 123; //OK !`
`  print!("{}", my_var);  //Prints "123"`
`}`
	`import static java.lang.System.out;`
	
	`public class Shadowing {`
	
	`  private static String myVar = "Hello World";`
	
	`  public static void main(String [] args) {`
	`  { //New scope`
	`  int myVar = 123;  //New declaration`
	`  out.print(myVar); //Prints "123"`
	`  }`
	`  out.print(myVar); //Prints "Hello World"`
	`  }`
	`}`

Rust	**Java**

Come si evince dall'esempio sopra riportato, a differenza di Rust, Java continua a mantenere il riferimento alla dichiarazione originale forzando la creazione di più scope per attuare lo shadowing o, nel caso di variabili non statiche, l'utilizzo del nome di riferimento completo (ad esempio, tramite la parola chiave *"this"*).

In Rust, l'immutabilità di una variabile può anche essere espressa tramite l'utilizzo della parola chiave *"const"* che ne impedisce lo shadowing e qualsivoglia ridenominazione. Il valore, e di conseguenza il tipo, contenuto nella costante deve essere indicato all'atto della dichiarazione di quest'ultima poiché esso verrà considerato non modificabile dal compilatore: motivo per cui le costanti non possono riferire, ovvero contenere nella dichiarazione, alcun valore mutabile. Rust offre quindi la possibilità di creare tre varianti diverse di ogni variabile (costante, dal valore immutabile o interamente mutabile) permettendo una maggiore flessibilità nella stesura del codice e conferendo al compilatore l'abilità di effettuare ottimizzazioni specifiche.

Negli esempi sopra riportati vengono utilizzate la parole chiave *"print!"* e *"println!"* che fanno parte di quella categoria di funzioni Rust in grado di avere un comportamento unico a seconda del tipo di parametri passati (espressioni, blocco di istruzioni, funzioni, pattern e molti altri). Queste funzioni particolari vengono chiamate *"macro"* e terminano sempre per *"!"*.

Nonostante Rust non supporti i tipi di dato dinamici [24] supportati da altri linguaggi di programmazione, come Javascript e Dart, è presente una forma di polimorfismo parametrico tramite l'utilizzo di tipi di dato generici all'interno delle definizioni di struct e funzioni.
Il tipo delle variabili *"generic"* non è un'eccezione al sistema di typing statico di Rust: esso viene controllato dal compilatore all'atto dell'istanziazione, garantendo quindi che ogni variabile ne abbia uno ben definito a compile-time evitando possibili errori dovuti ad incompatibilità o conversioni improprie effettuate a run-time.
Al fine di implementare un comportamento specifico a seconda del tipo di dato assunto dai *"generic"*, questi possono essere utilizzati in combinazione con i *"trait"*. Questi ultimi descrivono le funzionalità supportate da uno o più specifici tipi di dato e sono gli analoghi delle interfacce Java e C#, con la differenza però che essi non supportano alcun tipo di ereditarietà.

Di seguito viene riportato un esempio completo di implementazione di polimorfismo in Rust utilizzando *"trait"* e *"generic"*.

```rust
struct myStruct<T> { //Generic declaration
    myField: T
}
```

```rust
trait myTrait { //Trait declaration
    fn myTraitFunc(&self);
}

impl<T> myTrait for myStruct<T>
where T: Display { //Trait implementation
    fn myTraitFunc(&self) {
        println!("{}", self.myField);
    }
}
```

```rust
fn myFunc <T> (t: myStruct<T>) where T: Display {
    t.myTraitFunc(); //Trait use
}
```

```rust
fn main() {
    let my_var = myStruct {
        myField: "Hello World"
    };

    let my_other_var = myStruct {
        myField: 123
    };

    myFunc(my_var); //Prints "Hello World"
    myFunc(my_other_var); //Prints "123"
}
```

[24] https://en.wikipedia.org/wiki/Dynamic_programming_language

3.2 Gestione della memoria

Per quel che riguarda la gestione della memoria esistono in generale due tipi di approcci: il primo necessita che lo sviluppatore si occupi manualmente della gestione tramite gli strumenti di basso livello forniti dal linguaggio utilizzato (come succede in C); il secondo mira all'automazione di questo processo escludendo completamente l'intervento dello sviluppatore e delegando questo compito ad un componente esterno fornito dall'ambiente di esecuzione: il Garbage Collector.
Il primo approccio fornisce solitamente prestazioni di gran lunga superiori al secondo in cambio di tempi di manutenzione e dispendio di risorse maggiori. L'utilizzo di un Garbage Collector permette invece di ridurre i tempi di sviluppo e manutenzione del software riducendo le prestazioni di quest'ultimo a causa del fatto che durante le operazioni di analisi e pulizia della memoria l'esecuzione del programma viene completamente interrotta.
Rust adotta un approccio intermedio introducendo un insieme regole che lo sviluppatore è tenuto a rispettare al fine di garantire che il codice che ha scritto sia effettivamente sicuro dal punto di vista della memoria, rimuovendo la necessità di avere un Garbage Collector ed al contempo limitando le operazioni di accesso e modifica della memoria da parte dello sviluppatore.

In Rust il compito di controllare la correttezza del codice e di assicurarsi che quest'ultimo rispetti le regole di *"memory safety"* è delegato al compilatore. Quest'ultimo infatti controlla, a compile-time, che i principi di *"ownership"* e *"borrowing"* siano rispettati.
Il primo di questi criteri afferma che in Rust ogni valore è contenuto all'interno di una variabile che viene definita come *"owner"* (o proprietario) del valore e che deve essere unica in ogni istante di tempo. Inoltre secondo il principio di *"ownership"*, quando una variabile *"owner"* esce dallo scope corrente il suo valore viene rilasciato ed essa viene deallocata e distrutta.
È il compilatore che si occupa di questa fase di liberazione delle risorse tramite l'invocazione automatica della funzione *"drop"* [25].

Di seguito viene riportato un esempio di utilizzo della regola di *"borrowing"* e delle conseguenze che quest'ultima ha sull'utilizzo delle variabili.

---

[25] https://doc.rust-lang.org/std/ops/trait.Drop.html

```rust
fn main() {
    let my_var = String::from("Hello World");  //Original owner
    let my_other_var = my_var;          //New owner of the string
    println!("{}", my_other_var);       //Prints "Hello World"
    println!("{}", my_var);             //ERR! It's not the owner aymore
} //Drop function called for "my_var" and "my_other_var"
```

Ownership Rule

Le regole di *"ownership"* di Rust vengono spesso associate al principio di *"borrowing"* che permette di prendere letteralmente in prestito una variabile tramite il suo riferimento, eseguire delle operazioni con essa, e poi restituire il valore al precedente proprietario. Una parte specifica del compilatore Rust, chiamata *"borrow checker"*, si assicura che in un preciso istante di tempo ci sia esclusivamente un solo riferimento mutabile oppure un numero illimitato di riferimenti immutabili ad una variabile *"owner"* di un valore.
Questa semplice regola è alla base di un potente principio: in Rust non esistono data races ad una stessa risorsa poiché questa può avere solo ed esclusivamente un proprietario alla volta ed ogni altro riferimento ad essa o è immutabile, e quindi non modifica il valore della risorsa, oppure è mutabile ed è esattamente l'unico riferimento in grado di variare quel valore in quell'istante di tempo.

Nonostante abbiano lo stesso nome e una sintassi simile, i puntatori in Rust differiscono da quelli del linguaggio C poiché nel primo indicano il cambio momentaneo del proprietario, che comunque rimane unico, di un dato valore mentre nel secondo sono semplicemente la chiave di accesso all'area di memoria fisica in cui quel dato è memorizzato senza però garanzie che questa chiave sia unica. Inoltre dato che in Rust il tipo *"null"* non esiste, un riferimento non può mai assumere un valore ignoto o nullo a differenza dei puntatori del linguaggio C.

Il *"borrowing"* di una variabile viene effettuato tramite l'operatore *"&"*, mentre per l'accesso al valore puntato è necessario utilizzare il simbolo *"*"* dereferenziandone il riferimento. Quest'ultima operazione può essere omessa in tutti quei casi in cui il compilatore è in grado di inferirla autonomamente (si parla in questo caso di *"Deref coercion"*).

Di seguito viene riportato un esempio di *"borrowing"* di una variabile e delle conseguenti implicazioni che questa operazione ha.

```
fn myBorrower(my_borrow : &str) {
    let brwd = my_borrow;                  //Borrowing as immutable
    println!("Manual: {}", *brwd);         //Borrows "brwd" in order to print it
    println!("Auto: {}",       brwd);      //Automatic dereferencing

    let mut brwd_mut = my_borrow; //Tries to borrow as mutable
    brwd_mut.push('!'); //ERR! Cannot borrow as mutable, already borrowed as immutable
}
```

```
fn main() {
    let my_var = "Hello World";          //Original owner
    myBorrower(my_var);                  //Borrows the value
    println!("{}", my_var);              //OK! "my_var" is the owner  again
}
```

Borrowing Rule

Per garantire la corretta gestione della memoria, oltre ai due importanti principi appena descritti, Rust introduce il concetto di *"lifetime"* esplicito di una variabile.
Non è una nuova regola, bensì una nozione comune a tutti i linguaggi di programmazione: per *"lifetime"* di una variabile si intende il tempo di vita di quest'ultima. La differenza rispetto agli altri linguaggi è che il compilatore Rust richiede che, ogni qualvolta esso non sia in grado di garantire che un riferimento ad una determinata variabile sarà sempre accessibile all'interno dello scope entro il quale esso è stato dichiarato, il tempo di vita di questo riferimento venga reso esplicito.
È possibile esplicitare il tempo di vita di un riferimento tramite l'operatore *" 'lifetime_name"* dove per *"lifetime_name"* si intende di solito un singolo carattere, ad esempio: *" 'a"*, che nelle firme delle funzioni diventa *"<'a>"*.
Rendere esplicito il tempo di vita di un riferimento rende possibile rispondere a domande del tipo: "se A utilizza un riferimento a B per effettuare delle operazioni ma in alcuni casi questo riferimento viene invalidato, cosa si può dire in generale sul riferimento a B?".

Di seguito viene riportato un esempio di codice che permette di capire meglio l'importanza dei *"lifetime"* espliciti.

```
fn A(str1 : &str, str2: &str) {
    println!("{} {}", str1, str2);  //Should print "Hello World"
}
```

```
fn B() {
    let str1: &str = "Hello";
    let str2: &str = "World";
    thread::spawn(|str1, str2| { //New thread that borrows "str1" & "str2"
        thread::sleep(1000);      //A sleeps 1 second
        A(str1, str2);            //Will "str1" and "str2" still be valid ?
    };
    thread::sleep(1100);          //B sleeps 1.1 second
} //"str1" & "str2" are dropped
```

References' Lifetimes Problem

Rust distingue i *"lifetime"* in due tipi: gli *"input lifetimes"* e gli *"output lifetimes"*, che fanno rispettivamente riferimento ai parametri in ingresso e ai valori di ritorno delle funzioni.
Il motivo per cui Rust enfatizza in modo particolare i tempi di vita dei riferimenti in ingresso e in uscita dalle funzioni è che queste rappresentano la maggior parte dei casi in cui i *"lifetime"* sono rilevanti.
Sono infatti presenti regole di elisione specifiche per questi tipi di riferimenti che rendono la firme dei metodi meno verbose di quel che altrimenti sarebbero se i tempi di vita dovessero costantemente essere esplicitati.
In particolare le *"lifetimes elision rules"* specificano che ad ogni parametro, che è di tipo riferimento, venga assegnato un *"lifetime"* e che nel caso sia presente esattamente un *"input lifetime"* quest'ultimo venga assegnato a tutti gli *"output lifetime"*: in particolare se un *"input lifetime"* dichiara il tempo di vita del parametro *"self"*, tutti gli *"output lifetime"* vengono eguagliati a quel *"input lifetime"*.
In tutti gli altri i casi, non coperti da queste regole, è necessario esplicitare i tempi di vita dei riferimenti.

3.3 Runtime

Con il termine *"runtime"* si intende quell'insieme di funzioni e procedure che sono invocate durante l'esecuzione di un software e che ne permettono il corretto funzionamento.
Gli sviluppatori di Rust lo definiscono come un linguaggio *"with essentially no runtime system"*[26] ovvero *"senza un sistema di runtime"*.
Quest'affermazione non va ovviamente interpretata in senso letterale poiché qualsiasi software scritto in qualsiasi linguaggio di programmazione ha bisogno di un runtime per poter essere eseguito.
Quello che i creatori di Rust intendono è che il tempo di esecuzione delle procedure di inizializzazione e delle funzioni che fanno il binding delle operazioni essenziali al funzionamento del linguaggio stesso, è estremamente basso e perciò ignorabile nella maggior parte dei casi.
È però interessante osservare nel dettaglio cosa accade quando un programma Rust viene eseguito.
In particolare sono due le fasi principali di questo processo: quella di inizializzazione e quella di gestione dell'esecuzione.

La fase di inizializzazione [27] è di supporto all'esecuzione del main e i suoi scopi principali sono quelli di: definire il comportamento delle operazioni aritmetiche e logiche presenti nel crate [28] di librerie standard *"libstd"*, fornire il supporto sia ad interazioni con la memoria heap che alla gestione delle situazioni critiche generate dalla macro *"panic!"* in modo diretto o indiretto (ad esempio come conseguenza della primitiva *"unwrap"*), eseguire il main e restituire un intero che indica il risultato di queste operazioni.

Tutte queste funzionalità vengono implementate tramite la definizione della funzione *"start"* che viene a sua volta implementata utilizzando le funzioni *"lang_start"* e *"lang_start_internal"* definite nel crate *"rt"*.

---

[26] https://prev.rust-lang.org/en-US/faq.html
[27] https://github.com/rust-lang/rust/blob/a9ca1ec9280ca1e5020edd699917c3367a30a798/src/libstd/rt.rs
[28] https://doc.rust-lang.org/book/ch07-01-packages-and-crates.html

Le funzioni *"lang_start"* e *"lang_start_internal"* hanno 3 parametri di ingresso: un puntatore alla funzione main, un puntatore ad una sequenza di byte in memoria, che deve essere interpretata come una sequenza di stringhe, chiamata *"argv"* (l'analoga di quella C) ed un intero denominato *"argc"* (di tipo i32 o i64 bit a seconda del sistema operativo) che indica la dimensione di quell'area di memoria.

Di seguito viene riportato il codice (troncato) relativo a queste due funzioni.

```
#[cfg(not(test))]
#[lang = "start"]
fn lang_start<T: crate::process::Termination + 'static>(
    main: fn() -> T,
    argc: isize,
    argv: *const *const u8,
) -> isize {
    lang_start_internal(&move || main().report(), argc, argv)
}
```

```
#[cfg(not(test))]
fn lang_start_internal(
    main: &(dyn Fn() -> i32 + Sync + crate::panic::RefUnwindSafe),
    argc: isize,
    argv: *const *const u8,
) -> isize {...} //Implementation omitted
```

lang_start & lang_start_internal

La funzione *"lang_start_internal"* importa le definizioni dei componenti *"panic"*, *"system"* e *"thread"*, specificate nei rispettivi crate, relative a: componenti di basso livello, dipendenti dal sistema operativo in cui viene lanciato il programma, che potranno poi essere utilizzate da Rust (pipe, lock mutex, primitive I/O, ecc...); il comportamento del software nel caso in cui si dovesse verificare una situazione critica in cui l'esecuzione deve necessariamente essere interrotta (ripercorrere lo stack e liberare le aree di memoria corrispondenti, fornire un codice ed eventualmente un messaggio di errore, ignorare la deallocazione dello stack se specificato negli argomenti di invocazione, ecc...); l'implementazione dei thread Rust ed il legame con quelli del sistema operativo.

Successivamente viene invocato il metodo *"sys::init"* che si occupa della gestione, durante la fase di startup, del segnale *"SIGPIPE"* che altrimenti causerebbe, sui sistemi UNIX, un possibile crash del programma: su Windows il corpo della funzione è vuoto.

Di seguito viene riportato il codice relativo alle definizioni dei componenti appena discussi e all'implementazione del metodo *"sys::init"* su sistemi UNIX.

```
#[cfg(not(test))]
fn lang_start_internal(...) -> isize {
    /* Imports for "panic", "system" & "thread" components */
    use crate::panic;
    use crate::sys;
    use crate::sys_common;
    use crate::sys_common::thread_info;
    use crate::thread::Thread;

    sys::init();
    ...
}
```

Initial definitions

```
pub fn init() {
    unsafe {
        reset_sigpipe();
    }

    #[cfg(not(any(target_os = "emscripten", target_os = "fuchsia")))]
    unsafe fn reset_sigpipe() {
        assert!(signal(libc::SIGPIPE, libc::SIG_IGN) != libc::SIG_ERR);
    }

    #[cfg(any(target_os = "emscripten", target_os = "fuchsia"))]
    unsafe fn reset_sigpipe() {}
}
```

Unix implementation of sys::init

A questo punto viene definito il comportamento che il programma deve tenere nel caso di situazioni di stack overflow e viene poi istanziato il thread che eseguirà il codice del main. Quest'ultimo però non viene messo subito in esecuzione, ma viene prima eseguita una funzione di conversione degli argomenti di invocazione chiamata *"sys::args::init"* necessaria a causa del fatto che questi ultimi, su piattaforme GNU/Linux, vengono passati dalla libreria C *"glibc"* a Rust in un formato non standard chiamato *".init_array"*.
Su Windows e macOS questa funzione non è necessaria e il suo corpo è vuoto.
Seguendo le stesse politiche Rust per cui non è possibile avere puntatori mutabili ed immutabili che riferiscono una stessa area di memoria, gli argomenti di invocazione (che sono puntatori di puntatori a stringhe) vengono copiati, tramite la funzione *"args"*, utilizzando la primitiva *"clone"* definita nei crate importati in precedenza.
A questo punto il main viene finalmente eseguito e alla sua terminazione viene restituito, nel caso in cui non venissero riscontrati errori, il codice 101.

Di seguito viene riportato il codice relativo alle operazioni sopra riportate.

```
fn lang_start_internal(...) -> isize {
    ...
    unsafe {
        let main_guard = sys::thread::guard::init();
        sys::stack_overflow::init(); //Stack overflow management
        let thread = Thread::new(Some("main".to_owned()));
        thread_info::set(main_guard, thread);

        sys::args::init(argc, argv); //Internally calls the "args" function
        let exit_code = panic::catch_unwind(|| {
            sys_common::backtrace::__rust_begin_short_backtrace(move || main())
        }); //Main execution

        sys_common::cleanup(); //Deallocation of variables
        exit_code.unwrap_or(101) as isize //Exit code: 101 if successful
    }
}
```

Args convesion & Main execution

Il modello di esecuzione del runtime di Rust non prevede meccanismi di Garbage Collecting automatici per la gestione della memoria ma piuttosto segue la convenzione RAII (Resource Acquisition Is Initialization)[29] utilizzata anche in C++.

RAII è un idioma di programmazione basato sui concetti di stack e heap che prevede la creazione, chiamata *"initialization"*, delle risorse in concomitanza della loro allocazione, chiamata *"acquisition"*, tramite l'utilizzo di un costruttore e la loro deallocazione e distruzione, non appena queste risorse escono dallo scope in cui sono state dichiarate, tramite l'utilizzo di un distruttore.

Per *"stack"* si intende una struttura dati astratta che agisce come una collezione di elementi e sulla quale sono ammessi due soli tipi di operazioni: l'aggiunta (operazione di push) e la rimozione di un elemento (operazione di pop).
Essendo una struttura dati di tipo LIFO (Last In First Out) le operazioni di push e pop sono sempre eseguite a partire dalla cima dello stack.

Nei sistemi moderni, è possibile implementare uno stack utilizzando una parte della memoria RAM fisica e due registri general purpose presenti nelle CPU odierne chiamati *"Stack Base Register"* e *"Stack Pointer Register"*.
Dato che la memoria RAM può essere rappresentata come una lista di indirizzi, questa realizzazione è possibile poiché all'interno dei due registri vengono memorizzati l'indirizzo del primo e dell'ultimo elemento dello stack.
L'insieme di indirizzi compresi tra il primo e l'ultimo elemento di uno stack viene chiamato *"stack-frame"* e può contenere a sua volta uno o più *"stack-frame"*.

Col termine *"heap"* invece si intende quella porzione di memoria RAM dove risiedono le strutture dati dinamicamente allocate. A differenza delle strutture dati allocate sullo stack, quelle nella memoria heap non seguono un ordine di deallocazione stabilito a priori.

Nelle pagine seguenti si vuole analizzare in dettaglio la fase di gestione dell'esecuzione del runtime Rust utilizzando i concetti appena introdotti affiancati da piccoli esempi.

---

[29] https://en.wikipedia.org/wiki/Resource_acquisition_is_initialization

Si consideri ora il seguente codice Rust.

`fn main() {`
`  let x: i64 = 5;`
`  odd_square(x);`
`}`
	`fn odd_square(x: i64) -> i64 {`
	`  let y: &i64 = &x;`
	`  y * x`
	`}`

Come risultato della parte di inizializzazione del runtime antecedente all'esecuzione del main, troveremo all'interno dello *"Stack Base Registrer"* la posizione in memoria della nostra funzione main sotto forma di indirizzo.
Per comodità assumeremo che questo sia 0x0, che gli indirizzi siano tutti lunghi 64bit e che lo stack cresca da indirizzi *"bassi"* a indirizzi *"alti"*.
In questo caso osserviamo che per il main è necessario che siano allocati 64 bit per l'intero *"x"* in aggiunta ad altri 64 bit restituiti come risultato della funzione *"odd_square"*. Quindi lo *"stack-frame"* del main risulterà lungo 128bit: dall'indirizzo 0x0 al 0x2.
Notiamo inoltre che lo *"stack-frame"* del main contiene a sua volta lo *"stack-frame"* della funzione *"odd_square"*; quest'ultima è una funzione che prende in ingresso un argomento di tipo intero lungo 64 bit e che inizializza una variabile intera *"y"* della stessa dimensione per poi restituire il quadrato dell'intero passato in input sotto forma di i64: lo *"stack-frame"* di *"odd_square"* è perciò lungo 192 bit, ovvero da 0x3 a 0x5 (va precisato che il compilatore Rust utilizza un ulteriore indirizzo per salvare il dato di ritorno, come nell'esempio, solamente se impostato su *"debug mode"*).
A runtime, successivamente all'allocazione della variabile *"x"* da parte del suo costruttore, viene eseguita la funzione *"odd_square"* spostando lo *"Stack Base Register"* dall'indirizzo 0x1 all'indirizzo 0x3 (operazione di push sullo stack del main); viene poi invocato il costruttore della variabile *"y"* che ne inizializza il valore allocandola in memoria e spostando lo *"Stack pointer Register"* all'indirizzo 0x4. A seguire viene eseguita la moltiplicazione tra *"x"* e *"y"* salvando il valore di ritorno e deallocando la variabile *"y"* dalla memoria non appena questa esce dal suo scope di inizializzazione; viene infine restituito il valore della funzione *"odd_square"* al main (operazione di pop dello *"stack-frame"* relativo alla funzione *"odd_square"* dallo stack del main) riportando lo *"Stack Base Register"* al suo valore originale (in questo caso 0x0) e lo *"Stack Pointer Register"* al valore 0x2.

Si può osservare che nel modello RAII, senza alcun meccanismo di Garbage Collecting, le variabili inutilizzate vengono automaticamente deallocate, evitando memory leak e mantenendo elevate le performance.
I meccanismi proposti dalla filosofia RAII funzionano perfettamente anche nel caso in cui le dimensioni delle strutture dati non possano essere conosciute a priori dal compilatore.

Si prenda ora come esempio il seguente codice.

```
struct myStruct {
    value: i64
}

impl myStruct {
    fn new(value: i64) -> MyStruct {     //Definition of "myStruct" factory
        myStruct {
            value
        }
    }
}
```

```
fn odd_square_heap(x: i64) -> i64 {
    let y: myStruct = myStruct::new(x); //Use of "new" factory
    y.value * x
}

fn main() {
    let x: i64 = 5;
    odd_square_heap(x);
}
```

Restano valide le considerazioni fatte precedentemente con l'unica differenza che, in questo caso, la variabile *"y"* non può essere allocata sullo stack dato che la sua dimensione non è nota a priori dal compilatore Rust.
In questo caso quello che viene allocato sullo stack è un puntatore alla memoria heap in cui viene effettivamente memorizzata la sequenza di byte che rappresenta la struttura dati *"y"*.

3.4 Confronto Rust - Go - Java

L'utilizzo di Rust come base per lo sviluppo di un progetto software può essere giustificato da diverse ragioni, come la velocità di esecuzione, il ridotto consumo della memoria, la dimensione degli eseguibili prodotti e molte altre.
Le caratteristiche di questo linguaggio, in un contesto di scelta tecnologica, sono rilevanti solo se confrontate con quelle di quei linguaggi di programmazione che possono essere considerati come concorrenti di Rust.
In particolare è interessante paragonare le funzionalità di Rust con quelle di due colossi del mondo dello sviluppo software come Go e Java.

Go è un linguaggio di programmazione di basso livello, con typing statico, sviluppato da Google nel 2007 con lo scopo di fornire agli sviluppatori un linguaggio di programmazione semplice, veloce ed efficiente [30].
La sua sintassi è molto simile a quella del linguaggio C e, come quest'ultimo e Rust è uno dei linguaggi supportati da WebAssembly e che può quindi essere eseguito sul Web.

Java è un linguaggio di programmazione di alto livello, orientato agli oggetti e con typing statico, sviluppato da Sun Microsystems nel 1992 con lo scopo di dare la possibilità agli sviluppatori software di scrivere codice che non fosse in alcun modo dipendente dalla piattaforma su cui questo doveva poi essere messo in esecuzione. La filosofia di questo linguaggio di programmazione può essere riassunta dal celebre motto *"write once run anywhere"* [31].

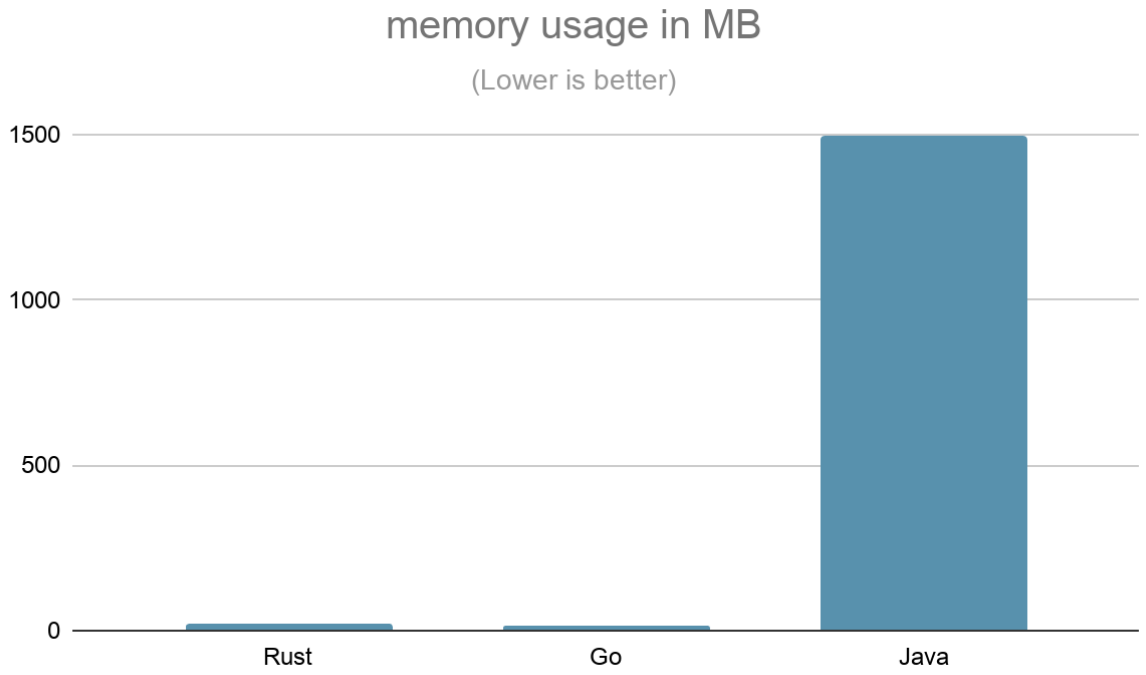
Si prenda ora come esempio un server Web il cui unico compito è quello di restituire una stringa in risposta a richieste HTTP di tipo GET realizzato rispettivamente in Rust, Java e Go.
Per comprendere meglio le differenze tra le tre versioni, sono di seguito riportati i risultati di alcuni test effettuati al fine di misurare alcuni parametri rilevanti, come ad esempio: throughput del server, consumo della memoria, dimensioni dei file ed altri.
Per effettuare queste prove è stato predisposto un pool di 400 thread che effettua continuamente richieste HTTP GET al server per un totale di 30 secondi. Il server alloca, serializza e restituisce la stringa *"Hello World"*.

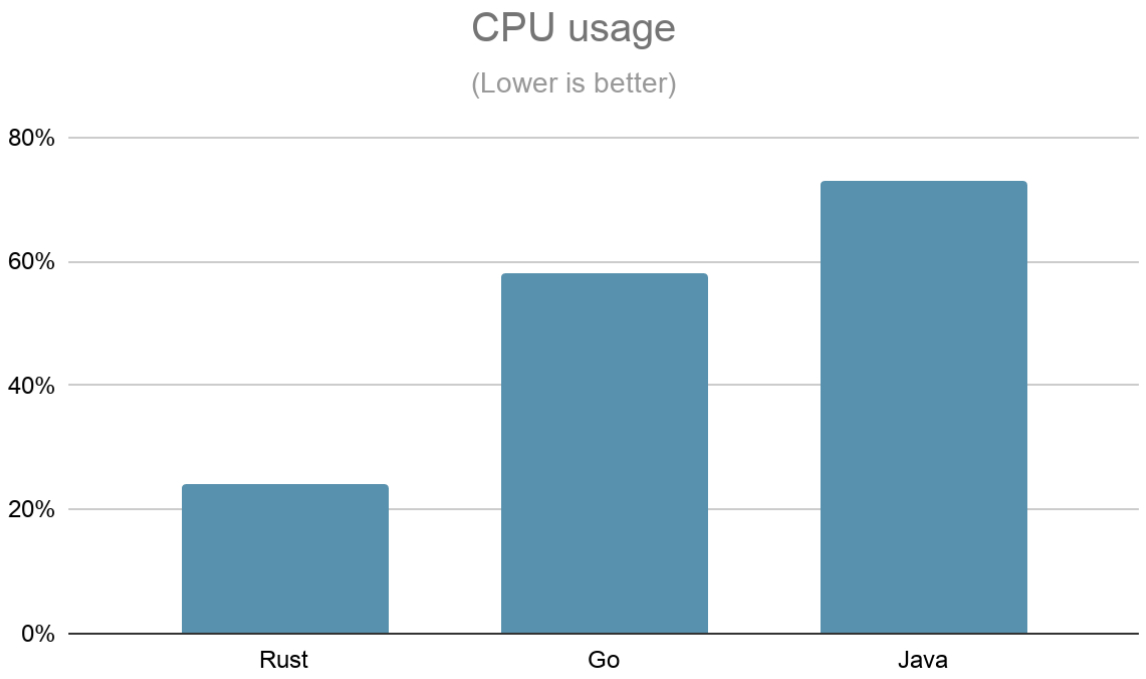
---

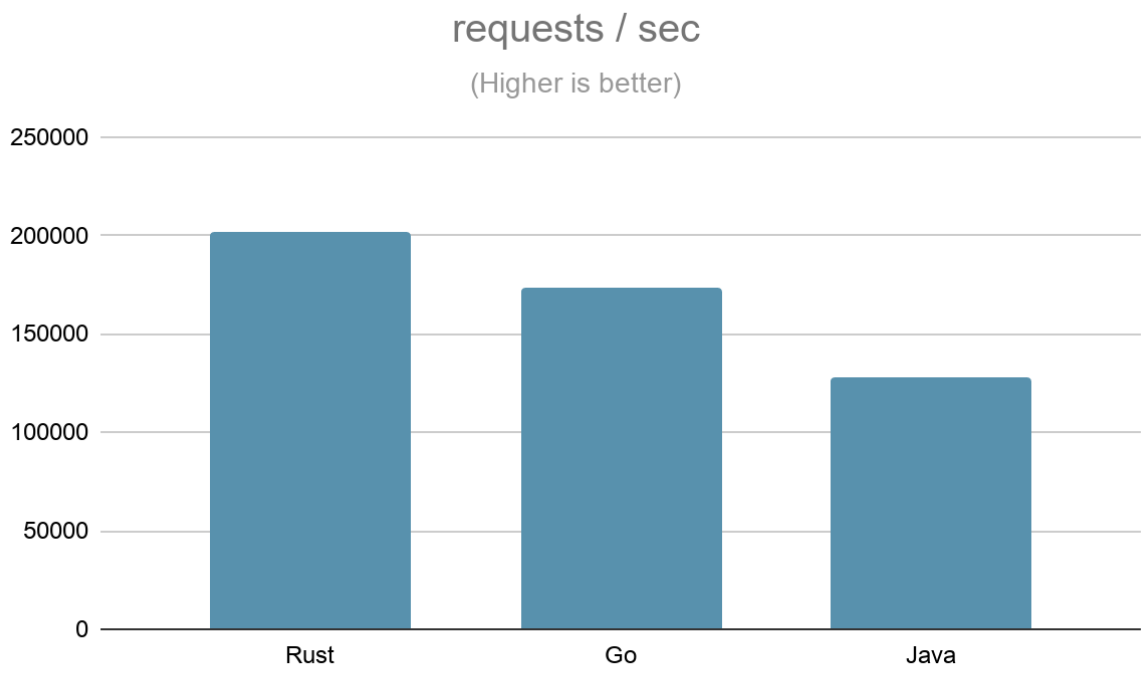
[30] https://github.com/golang/go/wiki
[31] https://en.wikipedia.org/wiki/Sun_Microsystems

Il primo test vuole misurare il consumo di memoria, l'utilizzo di CPU e la velocità di esecuzione pura dei tre linguaggi ed in particolare come questa influenza la velocità di risposta del server Web e la quantità di richieste che questo può servire in un determinato lasso di tempo, ovvero il throughput.

Di seguito i risultati ottenuti.







Dai test risulta che Rust utilizza la CPU il 49% in meno di rispetto a Java ed il 34% in meno rispetto a Go; inoltre Rust riesce a servire il 10% di richieste in più rispetto a Go ed il 36% in più rispetto a Java nello stesso ammontare di tempo.
Per quel che riguarda il consumo di memoria RAM, Rust e Go hanno ottenuto risultati simili, equivalenti a circa $\frac{1}{100}$ del consumo di memoria da parte della versione Java del server.
Questi test evidenziano che, in generale, Rust abbia performance superiori rispetto alle controparti Java e Go. Il motivo di questa differenza è dovuto al fatto che Java e Go utilizzano meccanismi di Garbage Collecting per gestire la memoria a tempo di esecuzione. Ciò implica che saranno inevitabilmente presenti momenti, non prevedibili a priori, in cui l'esecuzione dei programmi, o dei server in questo caso, sarà interrotta per dare la possibilità al Garbage Collector di eseguire le eventuali deallocazioni di variabili e strutture dati.
Come già discusso in precedenza, Rust segue invece la filosofia RAII, allocando e deallocando risorse solo quando strettamente necessario, rendendo le performance prevedibili e più consistenti.
Inoltre a differenza di Java, Rust non deve essere compilato in un bytecode intermedio per poi essere eseguito in una macchina virtuale dato che viene direttamente compilato in codice nativo.

Ciò implica che il codice Java avrà sempre ed inevitabilmente prestazioni inferiori ed eseguibili di dimensioni maggiori rispetto alla controparte Rust. Per quel che riguarda quest'ultimo aspetto, risulta che in generale gli eseguibili Rust hanno dimensioni inferiori rispetto a quelli Go. È inoltre possibile diminuire ulteriormente le dimensioni rimuovendo dal codice Rust tutte quelle funzionalità di debug, che erano presenti durante i test, tramite l'utilizzo di un flag speciale denominato *"--release"*.

Lo sviluppo di Rust è stato caratterizzato da continue scelte, da parte degli sviluppatori, in favore delle performance: basti pensare che non esistono data races tra thread ad una stessa risorsa per via del sistema di *"ownership & borrowing"* che invece sono presenti nelle controparti Java e Go e che vanno quindi gestite dallo sviluppatore.
Queste scelte sono chiaramente state fatte a discapito della facilità con cui è possibile scrivere il codice Rust: sviluppare in Java e Go risulterà estremamente più semplice rispetto alla controparte Rust; uno dei cavalli di battaglia di Go è proprio la semplicità con cui è possibile apprendere questo linguaggio e la velocità con cui è possibile cominciare produrre codice utile.

In tutti quegli ambiti in cui il tempo di sviluppo è limitato, l'utilizzo di linguaggi come Java e Go rimane quindi preferibile alla controparte Rust che, seppur migliorato rispetto alle prime versioni (nelle quali non esistevano le lifetime elision rules e la sintassi risultava piuttosto pesante e di difficile comprensione), ha ancora una curva di apprendimento piuttosto ripida.
Negli ambiti in cui le performance e la correttezza delle operazioni risulta cruciale invece, Rust risulta essere la scelta migliore.

Capitolo 4

Applicazione Web in Rust

Al fine di dimostrare le caratteristiche di Rust e WebAssembly, discusse nei capitoli precedenti, si vuole ora progettare una piccola applicazione Web scritta in Rust che utilizza il formato WASM per poter essere eseguita nel browser.
L'applicazione sarà una semplice *"chat room"* attraverso la quale sarà possibile inviare messaggi, in formato testuale, visibili da qualunque utente sia attualmente collegato al servizio; sarà inoltre possibile visualizzare la data e l'ora d'invio dei messaggi.
In aggiunta, si vuole che l'applicazione renda persistenti i messaggi fornendo una cronologia di questi ultimi al momento del collegamento di ogni utente.
Infine, per facilitare l'esecuzione di test, ogni connessione e disconnessione al servizio sarà monitorata.
Per semplicità non verranno implementate misure di sicurezza di alcun tipo.

Al fine di implementare componenti comuni nell'ambito della progettazione di applicazioni Web, senza dover riscrivere di volta in volta codice noto, si utilizzeranno delle librerie fornite dalla community di Rust, tra cui: *"chrono"* [32], *"WS-RS"* [33] e *"web_sys"* [34].

L'architettura del progetto sarà basata sul modello *"Client / Server"* [35] e verrà utilizzata la tecnologia *"WebSocket"* [36] al fine di poter garantire scalabilità ed efficacia nella comunicazione fra le parti.
Saranno quindi due le parti principali dell'applicazione: il server che si occupa della gestione dei messaggi ed il client frontend che si occupa dell'interazione con l'utente. Per comunicare utilizzeranno, seppur con implementazioni differenti, istanze di una stessa websocket attraverso cui invieranno stringhe rappresentati i messaggi scambiati dai client e le azioni da intraprendere.

---

[32] https://docs.rs/chrono/0.4.19/chrono/
[33] https://github.com/housleyjk/ws-rs
[34] https://rustwasm.github.io/wasm-bindgen/api/web_sys/
[35] https://en.wikipedia.org/wiki/Client%E2%80%93server_model
[36] https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API?retiredLocale=it

Per la realizzazione delle interfacce grafiche verrà utilizzato il framework *"Yew"* [37] al fine di facilitare il riutilizzo e la combinazione di componenti Web in maniera efficiente grazie anche all'integrazione nativa con WebAssembly.

"Yew" è una libreria Rust che permette la creazione di interfacce utente tramite l'utilizzo di componenti grafici integrati con HTML e Javascript; questa tecnologia si ispira fortemente alla ben più famosa controparte Javascript: *"React.js"* [38].
La scelta di questo framework è dovuta sia alle prestazioni che esso offre minimizzando le chiamate alle API DOM, di cui si è discusso nel capitolo 2.2, sia alla maturità e alla stabilità dimostrata se confrontato con alternative meno note.
In particolare si farà riferimento a quella parte di libreria che utilizza gli strumenti di compilazione *"wasm-bindgen"* [39] per permettere l'interoperabilità tra Javascript e Rust e, soprattutto, per esportare correttamente il codice Rust in formato WASM.

[37] https://yew.rs/docs/en/
[38] https://reactjs.org/
[39] https://github.com/rustwasm/wasm-bindgen

4.1 Backend

Il backend dell'applicazione è costituito un server che, una volta inizializzato, viene messo in ascolto di eventuali connessioni in ingresso sulla websocket. Per la realizzazione di quest'ultima si è deciso di utilizzare le funzionalità offerte dal crate *"WS-RS"*; in particolare questa libreria permette sia di creare una websocket e associarla ad un particolare indirizzo ip e ad una porta, sia di rispondere alla ricezione di eventi con callback personalizzati.
Questa libreria è di default single-threaded e ad ogni nuova connessione TCP viene associato un riferimento alla stessa websocket che risponderà, quando necessario, agli eventi generati. Quest'implementazione implica che ogni riferimento alle proprietà del server deve essere gestito manualmente dallo sviluppatore allo scopo di evitare incongruenze.

Per tenere traccia degli utenti attualmente collegati, si utilizza un contatore che deve poter essere letto e sovrascritto dal server ogni qualvolta un utente si connette o si disconnette.
Come discusso nel capitolo 3.1, in Rust non è possibile avere contemporaneamente riferimenti mutabili ed immutabili ad una stessa variabile; in questo caso però, è proprio quello di cui avremmo bisogno dato che molteplici utenti hanno riferimenti alla stessa websocket e quindi alle sue proprietà che vorremmo fossero lette o scritte in risposta a determinati eventi.
A tal fine, in Rust esistono delle strutture dati speciali chiamate *"Smart Pointer"* [40] che permettono, tramite l'utilizzo interno di codice *"unsafe"* [41], di avere contemporaneamente riferimenti mutabili ed immutabili ad una stessa variabile. Esistono molteplici versioni di questi wrapper e offrono tutte funzionalità differenti: nel nostro caso siamo interessati a *"Rc<T>"* [42] (Reference Counter di T) che permette di contare i riferimenti ad una stessa variabile.
Al fine di permettere la mutabilità interna, ovvero di mutare in modo atomico una variabile che ha già un riferimento immutabile, scegliamo come tipo generico *"<T>"* di *"Rc"*, *"Cell<Q>"* [43]: uno *"Smart Pointer"* che indica una locazione generica di memoria che può essere modificata.

---

[40] https://doc.rust-lang.org/book/ch15-00-smart-pointers.html
[41] https://doc.rust-lang.org/book/ch19-01-unsafe-rust.html
[42] https://doc.rust-lang.org/std/rc/struct.Rc.html
[43] https://doc.rust-lang.org/std/cell/struct.Cell.html

Di seguito viene riportato il codice relativo alla creazione e all'utilizzo del contatore di connessioni e disconnessioni.

```
struct Server { //Ws's server initial definition
  count: Rc<Cell<u32>>,
  ...
}

fn main() {
    let count = Rc::new(Cell::new(0));
    ...
}
```

```
fn on_open(&mut self, _: Handshake) -> Result<()> { //On new user connection
    ...
    Ok(self.count.set(self.count.get() + 1)) //Users connected +1
}

fn on_close(&mut self, code: CloseCode, reason: &str) { //On user disconnection
    ...
    self.count.set(self.count.get() - 1); //Users connected -1
}
```

Single-Threaded Interior Mutability

Al fine di visualizzare la data e l'ora dell'invio dei messaggi si è deciso di procedere in questo modo: accanto ad ogni messaggio viene visualizzata l'ora d'invio e, ogni mezzanotte, viene inviato un singolo messaggio, da parte del server a tutti gli utenti connessi, contenente la nuova data.
È semplice realizzare la prima funzionalità (dato che basta semplicemente manipolare le stringhe contenenti i messaggi ricevuti aggiungendo l'ora corrente alla fine) mentre più complicato è il caso della seconda.
Dato che vogliamo tenere traccia dell'ultima data nota al server è necessario che sia creata una struttura dati che la memorizzi e che periodicamente esegua l'operazione di aggiornamento; è inoltre necessario che la creazione di questa struttura avvenga una sola volta onde evitare errori.

Al fine di creare un'unica istanza del nostro timer creiamo una proprietà booleana del server, inizialmente impostata su *"false"*, che verrà cambiata solamente quando il timer verrà istanziato. Per le stesse ragioni per cui il contatore che tiene traccia del numero di utenti connessi deve essere utilizzato attraverso *"Rc<T>"*, è necessario incapsulare anche questa nuova proprietà del server nello stesso *"Smart Pointer"*. La creazione del nostro timer avverrà in concomitanza della prima connessione al server da parte di un qualsiasi client, in un thread separato da quello principale.
Per quel che riguarda l'aggiornamento periodico della data, viene utilizzata la libreria *"timer"* [44] (una derivata di *"chrono"*) che permette di impostare dei callback periodici.

Di seguito viene riportato il codice relativo alla creazione del timer e alla gestione dell'aggiornamento delle date dei messaggi.

```
struct Server { //Ws's server final definition
    count: Rc<Cell<u32>>,
    timer_created: Rc<Cell<bool>>,
    out: Sender //Ws's messages channel
}

pub struct Timer {
    tx: mpsc::Sender<String>,     //Used to trigger the callback
    rx: mpsc::Receiver<String>, //Used to respond to the callback
    out: Sender,                  //Used to send the new date to all users
    timer: timer::Timer,
    last_known_day: DateTime<Local>,
}

fn main() {
    let count = Rc::new(Cell::new(0));
    let timer_created = Rc::new(Cell::new(false));
    ...
}
```

New Definitions & Initializations

[44] https://github.com/Yoric/timer.rs

```rust
fn on_open(&mut self, _: Handshake) -> Result<()> { //On new user connection
    if self.timer_created.get() == false { //No timer has been created yet
        self.timer_created.set(true);

        let out = self.out.clone(); //Cloned because of lifetimes rules
        std::thread::spawn(move || { //Spawn new timer's thread
            let mut date_updater = Timer::new(out);
            let mut _guard = date_updater.update_date(date_updater.tx.clone());
            loop {
                let message = date_updater.rx.recv().unwrap(); //Wait till the callback
                if message == "update_date" { //Callback received
                    _guard = date_updater.update_date(date_updater.tx.clone());
                }
            }
        });
    }
    Ok(self.count.set(self.count.get() + 1)) //Users connected +1
}
```

```rust
fn update_date(&mut self, tx: mpsc::Sender<String>) -> timer::Guard {
    ... //Time offset calculation
    let millis_from_last_midnight = now_in_millis - last_known_midnight_in_millis;

    if millis_from_last_midnight > (24 * 60 * 60 * 1000) { //At least a day has passed
        ... //Setup a message with the new date
        self.out.broadcast(message).unwrap(); //Send the new date to all users
        self.last_known_day = now;                //Internally update the date
    }
    ... //Time offset calculation
    let millis_till_next_midnight = next_midnight_in_millis - now_in_millis;
    self.timer.schedule_with_delay( //Set a callback to be executed next midnight
        chrono::Duration::milliseconds(millis_till_next_midnight), move || {
            tx.send(String::from("update_date")).unwrap(); //Callback invocation
        })
  }
```

Timer creation & Callback Setup

Le ultime funzionalità da implementare lato server sono l'invio e la ricezione dei messaggi e la relativa persistenza.
Per semplicità implementeremo quest'ultima utilizzando un semplice file di testo, condiviso tra tutti gli utenti, in cui sarà memorizzata la cronologia completa dei messaggi della *"chat room"*.

Al fine di migliorare le performance, i compiti di gestione dei messaggi saranno affidati di volta in volta a thread differenti.
Questo però implica che un singolo file potrebbe venire utilizzato da più thread contemporaneamente ed il compilatore Rust non lo permetterebbe.
In questo caso non è possibile utilizzare lo *"Smart Pointer"* *"Rc<T>"* poiché esso può essere utilizzato solamente in contesti single-threaded.
Esiste una variante di *"Rc<T>"*, chiamata *"Arc<T>"* [45], dedicata agli ambiti multithreaded, ma in questo caso non è possibile utilizzarla dato che si andrebbe a condividere un file tra molteplici thread (avendo quindi un *"Arc<Cell<File>>"*), ma affinché in Rust una risorsa venga condivisa tra processi leggeri è necessario che questa implementi i trait *"Send"* [46] e *"Sync"* [47] che indicano al compilatore che una risorsa può essere condivisa in modo sicuro tra più thread: nonostante *"Arc<T>"* implementi *"Send"*, i file in Rust non implementano *"Sync"*.
Dobbiamo perciò ricorrere ad una variabile statica, che quindi abbia un lifetime pari a quello del server, che sia protetta da un lock per thread.
Per poterla definire è necessario utilizzare la macro *"lazy_static!"* che forza il tipo della nostra variabile ad essere un *"Mutex<File>"* [48] introducendo un lock alla risorsa: proprio quello di cui abbiamo bisogno.

Una volta creato un riferimento condiviso alla cronologia, possiamo utilizzare gli eventi della websocket per intercettare l'arrivo di nuovi messaggi e, a seconda del contenuto, decidere cosa fare.
Per agevolare lo svolgimento dei test, rendiamo esplicita, da parte di ogni client, la richiesta di ottenimento della cronologia dei messaggi e distinguiamo tra questo tipo di richiesta ed un normale messaggio tramite l'utilizzo di un prefisso che la parte frontend scriverà davanti ad ogni messaggio.

---

[45] https://doc.rust-lang.org/std/sync/struct.Arc.html
[46] https://doc.rust-lang.org/std/marker/trait.Send.html
[47] https://doc.rust-lang.org/std/marker/trait.Sync.html
[48] https://doc.rust-lang.org/std/sync/struct.Mutex.html

Di seguito viene riportato il codice relativo alla creazione del riferimento statico al file contenente la cronologia dei messaggi e quello relativo all'analisi e al broadcast dei messaggi.

```
lazy_static! {
    static ref CHAT_HISTORY_FILE: Mutex<File> = Mutex::new({
        OpenOptions::new()
                        .read(true)
                        .append(true)
                        .create(true)
                        .open("./resources/chat_history.txt")
                        .unwrap()
    });
}
```

```
fn on_message(&mut self, msg: Message) -> Result<()> {
    if msg.as_text().unwrap() == "get::" { //First connection from client
        println!("The number of live connections is {}", self.count.get());
        let out = self.out.clone();
        std::thread::spawn(move || {
            let mut file = CHAT_HISTORY_FILE.lock().unwrap();
            file.seek(SeekFrom::Start(0)).unwrap();    //Start from the beginning
            let lines = io::BufReader::new(file).lines(); //Read chat history file
            for line in lines { //Each line is a message
                match line {
                    Ok(message) => {
                        out.send(message).unwrap(); //Send the message to the user
                    }
                    Err(err) => {
                        println!("Error while sending chat history: {:?}", err);
                    }
                }
            }
        });
    } else ...
```

File's shared reference & History retrieval

```rust
fn on_message(&mut self, msg: Message) -> Result<()> {
    if msg.as_text().unwrap() == "get::" { //First connection from client
        ...
    } else { //Normal message received
        println!("message received: {}", msg);

        let out = self.out.clone();
        std::thread::spawn(move || {
            let now: DateTime<Local> = Local::now(); //Retrieve the current time
            ...//Message formatting
            let mut file = CHAT_HISTORY_FILE.lock().unwrap();
            if let Err(e) = writeln!(file, "{}", message) { //Update the chat history file
                println!("Cannot save chat history: {:?}", e);
            }
            out.broadcast(message).unwrap(); //Broadcast the message to all users
        });
    }
    Ok(())
}
```

Message Broadcast

Per quel che riguarda la parte server non rimane che la creazione della websocket stessa. Di seguito il codice relativo.

```rust
fn main() {
  let count = Rc::new(Cell::new(0));
  let timer_created = Rc::new(Cell::new(false));

  listen("localhost:8081", |out| Server { //Listen for new connections
    out: out,
    count: count.clone(),
    timer_created: timer_created.clone(),
  }).unwrap()
}
```

Final Main

4.2 Frontend

Il client che si occupa dell'interazione con l'utente e della gestione dell'interfaccia grafica della *"chat room"* è realizzato attraverso l'utilizzo del framework *"Yew"*.
Quest'ultimo contiene una libreria scritta in Rust che espone delle API tramite le quali è possibile interagire con il browser: l'equivalente delle API DOM di Javascript.
La libreria è basata sul modello a componenti e fa largo uso di eventi e callback per permettere l'aggiornamento di un eventuale stato interno dei moduli.
In particolare è possibile eseguire azioni specifiche ogni qualvolta un componente viene creato o aggiornato e quando questo riceve dei messaggi.

È inoltre possibile fare uso della macro *"html!"* per scrivere, tramite l'utilizzo di una sintassi simile a quella JSX [49] utilizzata in *"React.js"*, del codice HTML che costituirà poi il corpo del componente .

Per l'interazione con il browser, *"Yew"* si appoggia ad una libreria chiamata *"web_sys"* in cui sono presenti le implementazioni Rust della maggior parte delle funzioni della libreria standard di Javascript.
Infine, il codice prodotto viene automaticamente convertito nel formato WebAssembly tramite uno strumento di compilazione del codice Rust chiamato *"wasm_bindgen"*.

Per semplicità il frontend dell'applicazione avrà solamente un componente che rappresenterà sia la lista dei messaggi sia l'insieme degli input necessari ad un utente per digitarne il testo ed inviarne uno.
Normalmente i componenti *"Yew"* che verrebbero creati sono due: uno per la cronologia dei messaggi ed un altro per l'interazione con l'utente; in questo caso però il contesto è estremamente semplice e non ci sono reali vantaggi nel separare i ruoli o nello scrivere una logica specifica per la comunicazione tra essi. Utilizzeremo quindi i tag HTML per realizzare la struttura della pagina Web e sfrutteremo la logica a componenti basata su eventi di *"Yew"* per realizzare l'interattività richiesta.

---

[49] https://reactjs.org/docs/introducing-jsx.html

Al fine di implementare un qualsiasi componente che faccia uso di callback, in *"Yew"*, è necessario creare una struct che mantenga i riferimenti al contesto della pagina; nel nostro caso è necessario avere anche un riferimento alla websocket che utilizzeremo per scambiare messaggi con gli altri utenti, quindi la struct avrà un'ulteriore proprietà che rappresenterà proprio quest'ultimo riferimento.
È inoltre opportuno definire un enumerativo *"UserEvent"* che rappresenti tutti i tipi di messaggio che possono essere gestiti dalla logica del client frontend in modo da evitare conversioni ed uguaglianze scomode tra stringhe in seguito.
In questo caso gli eventi a cui siamo interessati sono quelli relativi alla tastiera e al click del mouse: in particolare i primi ci interessano solo quando viene premuto il tasto *"INVIO"* e l'utente sta digitando all'interno della casella di testo, mentre ci occupiamo dei secondi solo qualora avvenissero all'interno dell'area del bottone per l'invio dei messaggi.
Sviscerati tutti i casi d'interesse relativi alla parte d'interazione con l'utente, possiamo concludere che l'enumerativo annunciato poco sopra avrà solo due valori che chiameremo rispettivamente *"SendMessage"* e *"Ignore"*.
Dato che tramite la macro *"html!"* è possibile scrivere codice HTML, utilizzeremo quest'ultimo per agganciare gli event handler solo agli eventi opportuni: con questo metodo non possiamo però realizzare la condizione per cui una pressione sulla tastiera va ignorata se non relativa al tasto *"INVIO"*.
In *"Yew"*, al fine di poter rispondere a qualsiasi evento, è necessario che il componente principale abbia un riferimento a sé stesso ed implementi il metodo *"update"*. Nel nostro caso alla pressione del tasto *"INVIO"* o qualora l'utente cliccasse sul pulsante di invio del messaggio verrà generato un evento indirizzato al componente stesso scatenando l'invocazione del metodo *"update"* : all'interno di questo metodo verrà analizzato l'evento e verrà presa una decisione in merito all'azione da intraprendere risolvendo il problema sopra citato.
Il trait *"Component",* offerto da *"Yew"*, permette di utilizzare gli *"alias"* [50] di Rust per dare la definizione del tipo *"Message"* che ogni componente incorpora.
In pratica, *"Yew"* dichiara per noi il tipo *"Message"* e lo utilizza all'interno del metodo *"update"* per capire quale evento sia avvenuto senza darne una definizione. Utilizzando la keyword *"type"* di Rust possiamo definire *"Message"* esattamente come il nostro enumerativo *"UserEvent"* creando eventi personalizzati.

---

[50] https://doc.rust-lang.org/reference/items/type-aliases.html

Di seguito viene riportato il codice relativo all'enumerativo legato alle possibili tipologie di messaggio e alla macro *"html!"*.

```
pub struct ChatComponent {...}
pub enum UserEvent {
    Ignore,
    SendMessage
}

impl Component for ChatComponent {
    ...
    type Message = UserEvent; //From now on, Message is defined as UserEvent
    fn view(&self) -> Html {
        let send_callback = self.link.callback(|_: MouseEvent| UserEvent::SendMessage);
        let send_callback_on_enter = self.link.callback(|key: KeyboardEvent| {
            if key.key() == "Enter" { //User press "Enter" on the keyboard
                UserEvent::SendMessage
            } else { //User is still typing the message
                UserEvent::Ignore
            }
        });

    html! {
      ...
          <input ... onkeyup=send_callback_on_enter/>
          <button ... onclick=send_callback>SEND!</button>
      ...
    }
  }
}
```

html! & Events aliases

Al fine di poter inviare e ricevere messaggi è necessario che, esattamente come fatto nella parte di backend, si utilizzi una websocket come canale di comunicazione tra i vari client.
A differenza di quanto realizzato nella parte del server però, non è possibile utilizzare il crate *"WS-RS"* poiché le funzioni contenute all'interno di quest'ultimo non sono supportate da *"wasm-bindgen"* e quindi il compilatore Rust non saprebbe come tradurle in WASM. Ancora una volta quindi, facciamo affidamento alle libreria *"web_sys"* che invece è supportata dallo strumento di compilazione per WebAssembly ed offre un'implementazione nativa delle websocket Javascript utilizzabili in Rust.
Sfruttando nuovamente la logica ad eventi, approfittiamo del metodo *"create"*, che definisce il comportamento del costruttore del nostro componente *"ChatComponent"*, per creare la websocket e inizializzare le funzioni da richiamare in risposta agli eventi di nostro interesse.
In particolare sono due gli eventi a cui siamo interessati: la creazione della websocket e l'arrivo di un messaggio.
Nel primo caso, come da specifica, possiamo sfruttare la connessione appena creata per contattare il server e richiedere, tramite l'inserimento di un identificatore speciale all'interno del messaggio, la cronologia completa della *"chat room"*. Qualora l'azione avesse successo, il server comincerebbe ad inviare al client i vari messaggi in suo possesso ed è opportuno definire quindi una funzione di callback da invocare alla ricezione di ogni messaggio.
Dato che abbiamo utilizzato la macro *"html!"* per definire la struttura del componente, è ora possibile utilizzare, grazie alla libreria *"web_sys"*, le funzioni Javascript di manipolazione del DOM all'interno del callback invocato dalla websocket alla ricezione di un messaggio: in questo modo è possibile ottenere un riferimento al componente che si occupa di mostrare i messaggi a schermo e manipolarlo aggiungendo il nuovo messaggio alla lista.

L'implementazione delle websocket di *"web_sys"* richiede che i callback siano passati sotto forma di funzioni anonime. In Rust questo concetto è espresso dalle *"closure"* [51], ovvero funzioni speciali che non necessitano di una firma esplicita e che sono in grado di prendere possesso delle variabili utilizzate anche se dichiarate in uno scope diverso, tramite la keyword *"move"*, rispettando le regole di ownership ed evitando al contempo problemi legati all'incertezza sulla validità dei riferimenti per cui sarebbe necessario applicare le *"lifetime rules"*.

---

[51] https://doc.rust-lang.org/book/ch13-01-closures.html

Di seguito viene riportata l'implementazione del metodo *"create"*, all'interno del quale avviene la creazione della websocket e dei rispettivi callback.

```
fn create(_: Self::Properties, link: ComponentLink<Self>) -> Self {
    let url = "ws://localhost:8081";
    let websocket = web_sys::WebSocket::new(url).unwrap();
    websocket.set_binary_type(web_sys::BinaryType::Arraybuffer);
    let cloned_ws = websocket.clone();

    let onopen_callback = Closure::wrap(Box::new(move |_| { //On websocket creation
        let msg = "get::".to_owned(); //Special request message, starts with "get::"
        match cloned_ws.send_with_str(&msg) { //Send request for chat history
            Ok(()) => {}
            Err(err) => {
                console::log_1(&JsValue::from(
                                        format!("Cannot retrieve chat history: {:?}", err)));
            }
        }
    }) as Box<dyn FnMut(JsValue)>);

    let onmessage_callback = Closure::wrap(Box::new(move |event: MessageEvent| {
        if let Ok(new_message) = event.data().dyn_into::<js_sys::JsString>() {
            match new_message.as_string() {
                Some(message) => { //Whenever a new message is received
                    update_chat(message); //The chat is updated with the new message
                }
                _ => {}
            }
        } //Ignore the error
    }) as Box<dyn FnMut(MessageEvent)>);

    websocket.set_onopen(Some(onopen_callback.as_ref().unchecked_ref()));
    websocket.set_onmessage(Some(onmessage_callback.as_ref().unchecked_ref()));

    ChatComponent {...websocket } //The chat component can use the websocket
  }
```

Websocket creation & Callbacks definitions

Infine non resta che definire il metodo *"update"* all'interno del quale è possibile utilizzare il *"ChatComponent"* appena creato e le API *"web_sys"* per ottenere due riferimenti: uno alla casella di testo contenente il messaggio da inviare ed un altro alla websocket tramite la quale inviare il messaggio.

Proprio come avviene in *"React.js"*, in *"Yew"* è possibile controllare l'aggiornamento di un componente evitando di dover ricreare l'intero sottoalbero del DOM che lo contiene al fine di evitare sprechi di risorse. Nonostante non faccia molta differenza in questo caso, il valore di ritorno del metodo *"update"*, se pari a *"true"*, effettuerà per noi esclusivamente il repaint di *"ChatComponent"*.

Di seguito il codice relativo al metodo "update".

```
fn update(&mut self, msg: Self::Message) -> ShouldRender {
    match msg {
        UserEvent::SendMessage => { //User has pressed ENTER or clicked on send button
            let input = self.input_field.cast::<HtmlInputElement>().unwrap();
            let input_value = input.value();
            let message = input_value.trim();

            if message != "" { //Avoid sending an empty message
                let mut msg = "post::".to_owned(); //"post::" tells the server this is a new msg
                msg.push_str(&escape(message)); //Characters escaping to avoid problems
                match self.websocket.send_with_str(&msg) { //Send message to the server
                    Ok(()) => {
                        input.set_value(""); //Reset input field
                        input.focus().unwrap(); //Focus the input field
                    }
                    Err(err) => {...}
                }
            } return true; //Update the component because the input field has changed
        }
        UserEvent::Ignore => {return false;} //Don't update the component
    }
}
```

Update function & Messages Sending

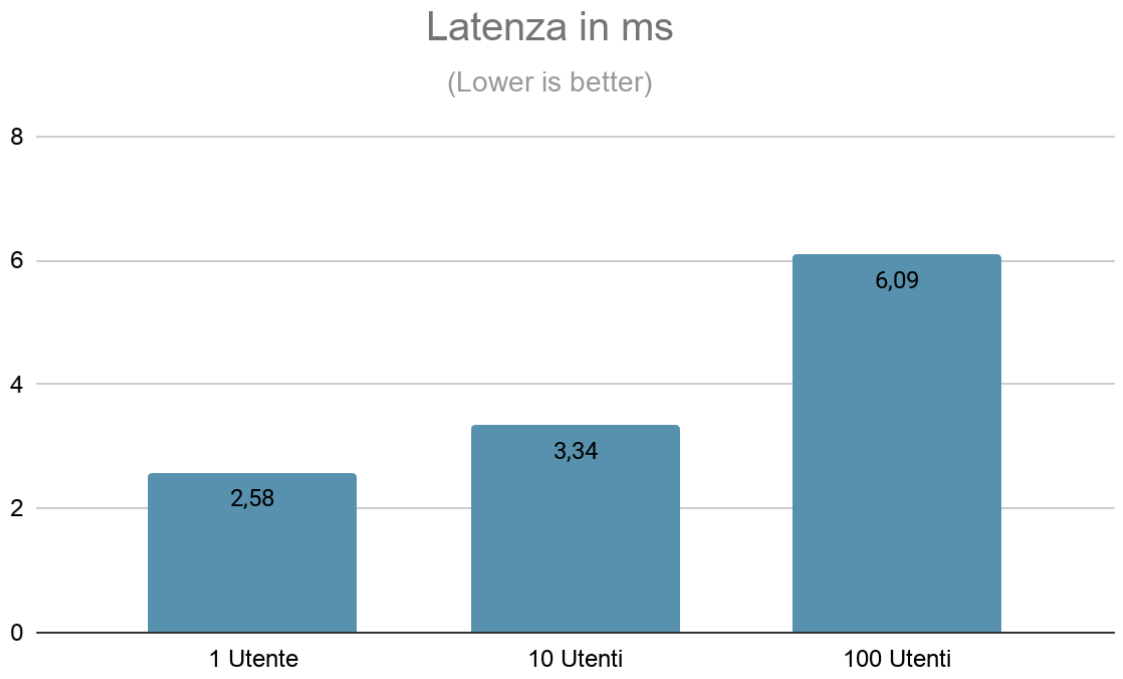
4.3 Esecuzione e Test

Una volta terminata la progettazione di tutte le parti che costituiscono la *"chat room"*, è interessante testarne il funzionamento analizzando i risultati.
Dato che l'obiettivo principale dell'applicazione è quello di permettere agli utenti collegati di scambiarsi velocemente messaggi di testo, sono due i principali indicatori di performance: il comportamento del sistema all'aumentare dei client collegati e all'aumentare dei messaggi inviati contemporaneamente.

Il modello di esecuzione del server web, che gestisce i messaggi, è quello di un processo master, in ascolto sulla porta 8081 tramite la websocket creata durante l'istanziazione, che crea un nuovo sotto-processo slave per ogni nuova connessione o messaggio ricevuto.
Questo modello permette la concorrenza delle operazioni ma, come già spiegato nel capitolo 4.1, introduce la necessità di sincronizzare gli accessi ad alcune risorse condivise.
In particolare, dato che abbiamo scelto di implementare la persistenza dei messaggi tramite l'utilizzo di un singolo file di testo, l'accesso alla variabile *"CHAT_HISTORY_FILE"*, che rappresenta il file contenente la cronologia dei messaggi, è stato reso sincrono tramite l'utilizzo di un lock per thread; ciò implica che solamente un processo alla volta può leggere o scrivere dal file.
Ovviamente questa soluzione non è ottimale e soprattutto limita molto la scalabilità dell'applicazione: l'utilizzo di più file o, ancora meglio, di un database per la realizzazione della persistenza è sicuramente una scelta migliore qualora il sistema dovesse necessitare di performance estremamente elevate.
Questo tipo di problema non è legato solamente al backend dell'applicazione ma anche alla parte frontend che si occupa dell'interazione con l'utente.
La semplice implementazione da noi adottata è di natura single threaded e sospende momentaneamente l'aggiornamento dell'interfaccia nel momento in cui vengono ricevuti o inviati dei messaggi: la risorsa condivisa, tra utente e websocket, è in questo caso l'interfaccia stessa. Sulla falsariga di quanto fatto nella parte server dell'applicazione, si potrebbero generare, tramite l'utilizzo delle API *"Yew"*, dei Web workers per la gestione di eventi comuni, come l'invio o la ricezione di messaggi dalla websocket.

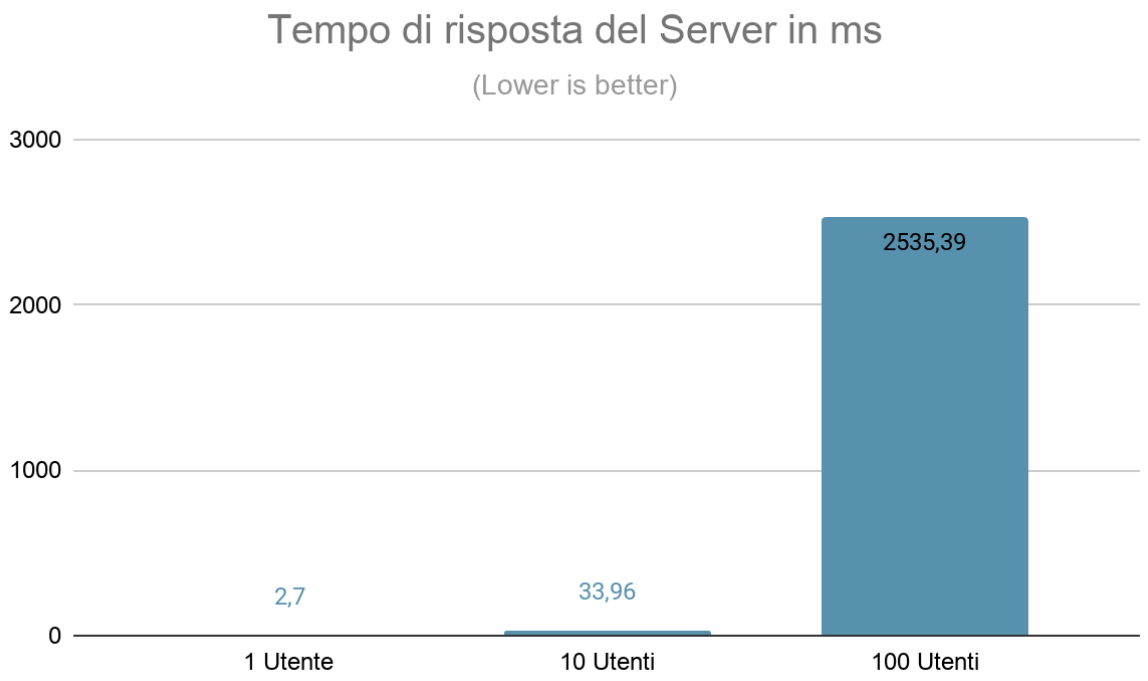
Nonostante lato server il problema della concorrenza sia evidente e limitante, lato utente questo diventa molto più tollerabile rispetto alla controparte backend poiché: ogni situazione in cui un singolo utente dovesse ricevere, lato frontend, un numero x di messaggi da altri y utenti, lato backend, il server dovrebbe comunque essere in grado di gestire x * y messaggi contemporaneamente. Inoltre una qualsiasi situazione in cui un utente dovesse ricevere contemporaneamente una quantità enorme di messaggi, renderebbe completamente inutilizzabile l'applicazione dal punto di vista pratico.
Per questi motivi i test verranno effettuati esclusivamente sulla parte server del sistema.

Il primo dei test effettuati riguarda il comportamento del server Web quando il numero di clienti collegati allo stesso tempo aumenta.
In particolare si vuole misurare il tempo che intercorre tra l'invio di un messaggio da parte di un client e la ricezione dello stesso da parte di un altro client. Di seguito i risultati.



Come si può osservare, l'incremento del numero di utenti collegati incrementa in maniera sub-lineare la latenza che gli utenti riscontrano nell'invio e nella ricezione dei messaggi. Ciò indica che eventuali colli di bottiglia non sono dovuti all'implementazione delle websocket utilizzata ma all'utilizzo che si è fatto di quest'ultima.

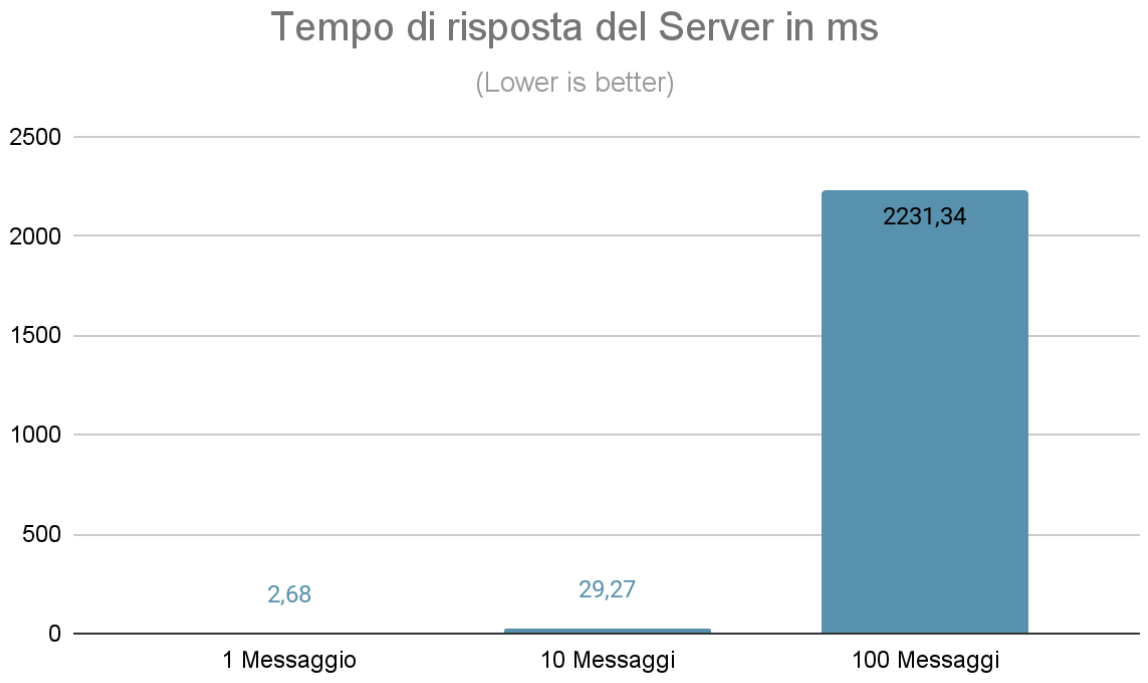
Per verificare quanto riportato dai primi test, effettuiamo un secondo controllo. Quest'ultimo però, differentemente da quanto fatto in precedenza, si focalizzerà sulla quantità di tempo impiegata da un singolo client frontend a ricevere risposta dal server in seguito alla richiesta di ottenimento dell'intera cronologia dei messaggi della *"chat room"* al crescere dei messaggi *"get::"*.
In pratica ci si chiede cosa possa succedere alle latenze se sempre più utenti richiedono contemporaneamente la cronologia dei messaggi al server.
Di seguito i risultati.



I risultati ottenuti indicano che i tempi di risposta crescono in maniera esponenziale. Ciò è perfettamente in linea con le considerazioni fatte poc'anzi ed il motivo va ricercato nel fatto che, al fine di ricevere i messaggi inviati, è necessario leggere il file che li contiene, ma quest'operazione è limitata da un lock e può essere effettuata esclusivamente in maniera sequenziale: in pratica, nonostante l'architettura del server sia multithreaded, dato che ogni iterazione di quest'ultimo con gli utenti fa uso di uno stesso file, il contesto diventa sequenziale e i tempi di attesa non beneficiano come dovrebbero dei processi leggeri creati.

Ragionamento analogo può essere fatto per quel che riguarda i tempi di risposta del server in seguito alla ricezione simultanea di molti messaggi di tipo *"post::"*.

Di seguito vengono riportati i risultati di un terzo test che vuole misurare i tempi di risposta del server all'aumentare dei messaggi ricevuti contemporaneamente.

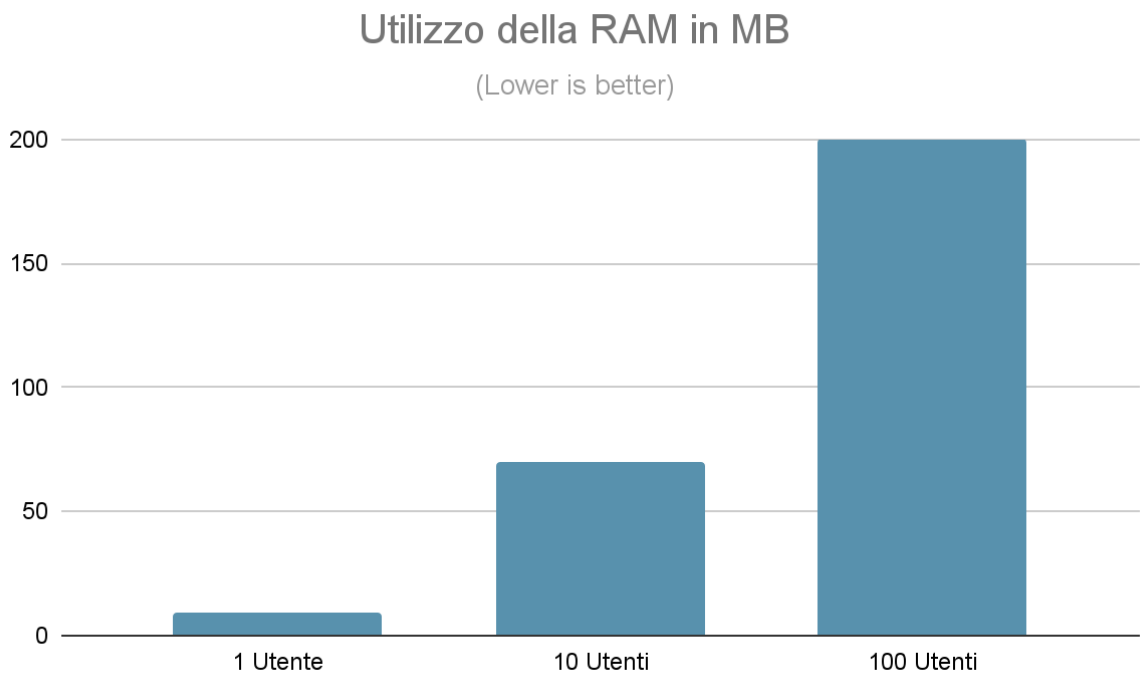


Anche in questo caso i tempi di risposta aumentano esponenzialmente e la ragione è di nuovo l'utilizzo della risorsa condivisa *"CHAT_HISTORY_FILE"*.

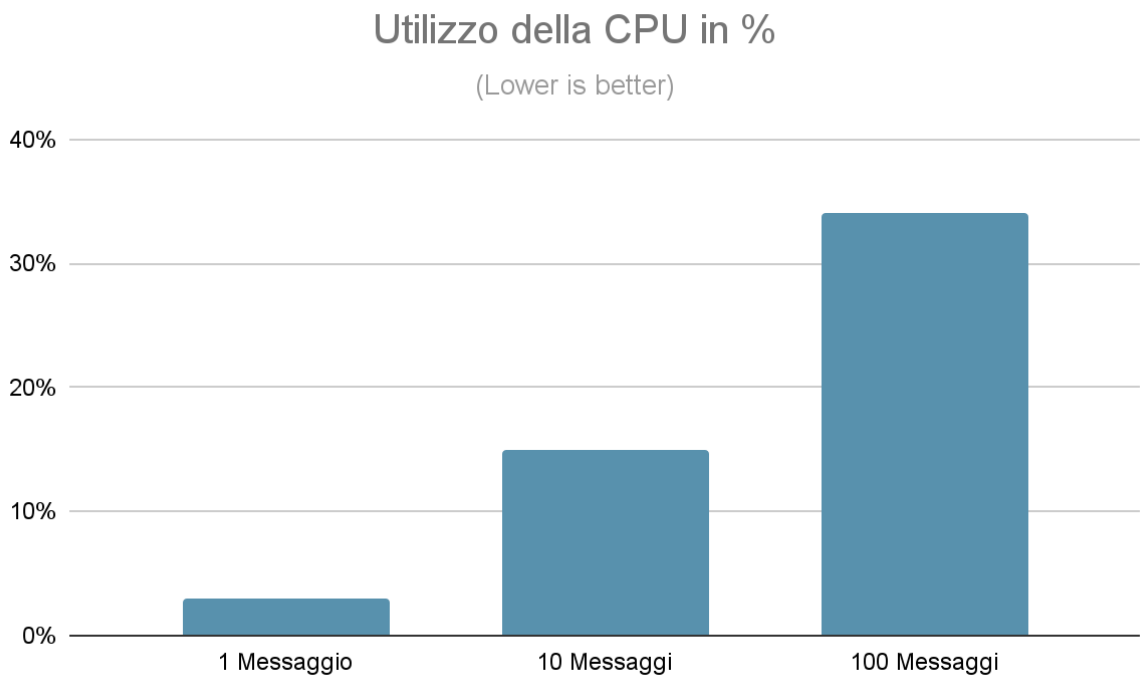
L'eventuale contributo, in senso negativo, apportato dall'utilizzo degli smart pointer *"Rc<Cell<T>>"* al fine di creare variabili condivise tra i thread del server, è pressoché irrilevante dato che si parlerebbe comunque di manipolazioni atomiche di variabili che impiegherebbero qualche ciclo di clock, i quali, se paragonati alle centinaia di migliaia utilizzati per la scrittura e la lettura su file, risulterebbero trascurabili.

Al fine di valutare quanto importante, in senso relativo, sia la limitazione imposta dall'utilizzo di un singolo file come metodo di persistenza, si vuole misurare l'utilizzo di CPU e RAM da parte del server all'aumentare di utenti connessi e di messaggi inviati contemporaneamente.
Questi ultimi due test sono meno indicativi delle performance assolute del sistema ma sono utili per capire come comportarsi, ad esempio, in situazioni in cui si deve valutare se il fattore limitante è l'hardware o il software.

Di seguito viene riportato il consumo di RAM da parte del server all'aumentare degli utenti connessi allo stesso tempo.



Di seguito vengono riportato l'utilizzo della CPU in situazioni di ricezione simultanea di messaggi da parte del server.



I risultati di questi ultimi due test dimostrano che, nonostante il sistema sia stato sottoposto ad uno sforzo intenso, le risorse hardware erano pienamente in grado di gestire la situazione; ora ogni dubbio è levato ed è possibile concludere che nonostante i meccanismi di memory-safety di Rust permettano di scrivere codice sicuro ed affidabile e il linguaggio sia per natura performante, abusare di queste caratteristiche e utilizzarle per realizzare velocemente funzionalità software che richiederebbero altresì una quantità notevole di tempo, porta comunque a problemi di affidabilità e performance.

D'altro canto però, anche se nessun tipo di ottimizzazione è stata impiegata, Rust si è dimostrato comunque un linguaggio competitivo ed in grado di utilizzare un quantitativo di risorse limitato: l'intera *"chat room"*, per essere eseguita, necessita di una quantità di risorse hardware paragonabili a quelle utilizzate da Java nel test di serializzazione ed invio della stringa *"Hello World"* effettuato nel capitolo 3.4.
La combinazione con *"Yew"* lato frontend ha inoltre evidenziato che, nonostante lo sviluppo ancora acerbo e lacunoso per alcuni aspetti, sia già possibile utilizzare Rust per la creazione in ambito professionale di vere e proprie applicazioni Web performanti e scalabili.